

Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA "TULLIO LEVI-CIVITA"

CORSO DI LAUREA IN INFORMATICA



**Sviluppo di una piattaforma evidence-based per
la prioritizzazione delle vulnerabilità mediante
analisi del rischio e contesto operativo**

Tesi di Laurea

Relatrice

Prof. Eleonora Losiouk

Laureando

Aldo Bettega

Matricola 2101087

«What I cannot create, I do not understand.»

— *Richard Feynman*

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage, della durata di circa trecento ore, dal laureando Aldo Bettega presso l'azienda Kirey Group. L'obiettivo del tirocinio è stato realizzare una piattaforma che affiancasse un analista di sicurezza nell'analisi di vulnerabilità informatiche.

Ringraziamenti

Innanzitutto, vorrei esprimere la mia gratitudine alla Prof. Eleonora Losiouk, relatrice della mia tesi, per l'aiuto e il sostegno fornitomi durante la stesura del lavoro.

Desidero ringraziare l'azienda Kirey Group e i professionisti con cui ho avuto il piacere di lavorare, i quali hanno reso possibile la creazione del progetto ThreatLens.

Un ringraziamento speciale va ai miei genitori per il fondamentale sostegno lungo questo percorso, e alla mia famiglia e ai miei fratelli per i momenti passati insieme. Ringrazio Maria per essermi stata sempre vicina e avermi supportato. Un ultimo pensiero è rivolto ai miei amici, per le significative esperienze condivise, e ai miei compagni universitari Marco e Paolo, per il prezioso aiuto in questi anni di studi.

Padova, Settembre 2026

Aldo Bettega

Indice

1. Introduzione	1.
1.1. L'azienda	1.
1.2. L'idea del progetto	1.
1.3. Organizzazione del testo	1.
2. Processi e metodologie	3.
2.1. Metodologia di lavoro	3.
2.2. Pianificazione delle attività	3.
2.2.1. Prima settimana	3.
2.2.2. Seconda settimana	3.
2.2.3. Terza settimana	3.
2.2.4. Quarta settimana	4.
2.2.5. Quinta settimana	4.
2.2.6. Sesta settimana	4.
2.2.7. Settima settimana	4.
2.2.8. Ottava settimana	4.
2.2.9. Nona settimana	5.
3. Studio del Dominio	7.
3.1. Limiti del CVSS	7.
3.2. Nuove metriche supportate dalla letteratura scientifica	7.
3.3. Il modello Vulnerability Management Chaining	8.
3.3.1. Analisi teorica del funzionamento	9.
3.3.2. Parametrizzazione dei valori	10.
3.3.3. Ruolo del contesto dell'asset	10.
4. Analisi dei Requisiti	11.
4.1. Casi d'uso del sistema	11.
4.2. Requisiti Funzionali	16.
5. Tecnologie	17.
5.1. Tecnologie di Backend	17.
5.2. Tecnologie di Frontend	18.
5.3. Strumenti di Sviluppo e Ambiente	19.
6. Ambiente di test	21.
6.1. Architettura del laboratorio	21.
6.2. Problemi e soluzioni adottate	21.
7. Progettazione Architetturale	23.
7.1. Architettura di backend _G : Ports & Adapters	23.
7.1.1. Diagramma delle classi di backend _G	24.
7.1.2. Inbound Adapters	24.
7.1.3. Servizi Applicativi	25.
7.1.3.1. AssessmentApplicationService	25.
7.1.3.2. PriorityEngine	26.

7.1.4. Outbound Adapters	26.
7.1.4.1. Scanner	26.
7.1.4.2. AnalysisStore	26.
7.1.4.3. Data providers	26.
7.1.4.4. AiAdapter	27.
7.2. Principi di Design e Modularità nel backend _G	27.
7.2.1. Inversione delle dipendenze	27.
7.2.2. Estensibilità tramite Strategy e Registry	27.
7.2.3. Single Responsibility Principle	28.
7.3. Modellazione della pipeline _G di assessment	28.
7.3.1. Fase 1: Scansione	28.
7.3.2. Fase 2: Enrichment	29.
7.3.3. Fase 3: Calcolo della priorità	29.
7.3.3.1. Fase 4: Generazione della spiegazione con l'AI _G	29.
7.3.3.2. Fase 5: Creazione del report	29.
7.3.4. Architettura di gestione degli errori	29.
7.3.4.1. Eccezioni Applicative	30.
7.3.4.2. Fallimenti di pipeline _G	30.
7.3.4.3. Guasti infrastrutturali	30.
7.3.4.4. Traduzione verso l'interfaccia	30.
7.4. Architettura di frontend _G	31.
7.4.1. Service Model	31.
7.4.1.1. AnalysisApi	31.
7.4.1.2. CapabilitiesApi	32.
7.4.1.3. PipelineApi	32.
7.4.1.4. ReportApi	32.
7.4.2. Facade	32.
7.4.2.1. HomeFacade	32.
7.4.2.2. NewAnalysisFacade	32.
7.4.2.3. ReportFacade	33.
7.4.3. Smart Component (ViewModel)	33.
7.4.4. Dumb Component (View)	33.
7.4.5. Flussi Asincroni e Gestione dello Stato Reattivo	33.
8. Implementazione e Scelte Tecnologiche	35.
8.1. Modellazione e Validazione dei Dati	35.
8.2. Persistenza dei dati nell'MVP _G	35.
8.3. Sicurezza e comunicazione HTTP	35.
8.4. Richieste API _G Batch	36.
9. Integrazione dell'Intelligenza Artificiale	39.
9.1. Strutturazione del Contesto e prompt _G Engineering	39.
9.2. Analisi dell'Allineamento	39.
9.3. Vincoli di Output e Prevenzione delle allucinazioni _G	40.
10. Conclusioni	41.
11. Glossario	42.
Bibliografia	45.

Elenco delle Figure

Figura 1	Logo di Kirey Group	1.
Figura 2	<i>Vulnerability Management Chaining Decision Tree</i> , da [1]	8.
Figura 3	Use Case - UC1: Creazione e avvio dell'analisi di sicurezza	11.
Figura 4	Use Case - UC2: Monitoraggio dell'avanzamento dell'analisi	12.
Figura 5	Use Case - UC3: Apertura e consultazione del report di vulnerabilità	13.
Figura 6	Use Case - UC4: Esportazione del report	14.
Figura 7	Use Case - UC5: Invio del report tramite email	15.
Figura 8	Diagramma architetturale dei componenti	24.

Elenco delle Tabelle

Tabella 1	Confronto delle metriche di prioritizzazione: Efficienza e Copertura in base al metodo adottato [1].	8.
Tabella 2	Regole decisionali e condizioni logiche per l'assegnazione delle classi di priorità.	9.
Tabella 3	Matrice di calcolo deterministico per il livello di attenzione contestuale (<i>Context Attention</i>).	10.
Tabella 4	Elenco dei Requisiti Funzionali obbligatori estratti dai casi d'uso.	16.
Tabella 5	Tecnologie di Backend	17.
Tabella 6	Tecnologie di frontend	19.
Tabella 7	Strumenti di sviluppo e ambiente	19.

Capitolo 1.

Introduzione

1.1. L'azienda



Figura 1: Logo di Kirey Group

Il tirocinio si è svolto presso *Kirey Group*, uno dei principali gruppi europei attivi nella consulenza tecnologica e IT, che affianca le aziende nel loro percorso di trasformazione digitale e di innovazione data-driven. In particolare, l'attività formativa si è inserita all'interno della divisione di Cybersecurity, specializzata nel fornire supporto e soluzioni avanzate per aiutare le organizzazioni a proteggersi dalle crescenti minacce informatiche.

1.2. L'idea del progetto

Il progetto intende inserirsi nel contesto del *Vulnerability Assessment_G*, un campo nel quale è presente il problema della *Vulnerability Fatigue / Alert Fatigue_G*. In tale scenario, la scoperta e la risoluzione di minacce rappresentano un processo continuo senza sosta. Questa dinamica porta ad un inevitabile **sovraccarico cognitivo** per chi deve assicurarsi che i sistemi siano aggiornati e superino un livello minimo di sicurezza. Per questo motivo occorre ottimizzare l'efficacia delle risorse allocate nella fase di *remediation_G*, intervenendo con priorità.

La piattaforma ha quindi l'obiettivo di affiancare l'analista di sicurezza nell'individuazione e nella gestione delle vulnerabilità più importanti, fornendo un contesto completo e attendibile.

1.3. Organizzazione del testo

- **Secondo Capitolo - Processi e Metodologie:** espone il metodo di lavoro adottato durante il tirocinio e le attività che si sono svolte di settimana in settimana.
- **Terzo Capitolo - Studio del Dominio:** analizza il problema di partenza ed espone le soluzioni adottate dal sistema, facendo riferimento alla letteratura scientifica del settore.
- **Quarto Capitolo - Analisi dei Requisiti:** riporta in modo dettagliato i casi d'uso individuati e i conseguenti requisiti previsti per il sistema.
- **Quinto Capitolo - Tecnologie:** elenca tutti gli strumenti e le tecnologie adottati per realizzare il sistema, tra cui linguaggi di programmazione, librerie e software.
- **Sesto Capitolo - Ambiente di Test:** approfondisce il laboratorio virtuale creato per testare il sistema.
- **Settimo Capitolo - Progettazione Architettuale:** espone rigorosamente tutta l'architettura di sistema, mostrando come è stata modellata e come sono state suddivise le

responsabilità tra i vari moduli. Fornisce un'idea superficiale del funzionamento del prodotto software.

- **Ottavo Capitolo - Implementazione e Scelte Tecnologiche:** spiega nel dettaglio le scelte implementative effettuate durante la fase di codifica del sistema.
- **Nono Capitolo - Integrazione dell'Intelligenza Artificiale:** approfondisce il ruolo di un modello linguistico di grandi dimensioni all'interno della pipeline.
- **Decimo Capitolo - Conclusioni:** riassume i risultati raggiunti e le conoscenze acquisite durante il periodo di tirocinio.

Capitolo 2.

Processi e metodologie

In questo capitolo viene riportato il modo in cui mi sono interfacciato con i tutor aziendali e come abbiamo organizzato il lavoro fissando i vari obiettivi lungo il corso del progetto.

2.1. Metodologia di lavoro

Durante le prime settimane l'attività si è svolta in modalità ibrida, per poi passare ad un regime completamente da remoto a causa dell'inagibilità della sede aziendale. Sebbene la modalità a distanza sia stata preponderante, l'efficienza operativa e la collaborazione non ne hanno risentito. L'uso di canali di comunicazione sincroni e asincroni come Teams_G ha garantito un aggiornamento continuo sull'avanzamento delle attività e un supporto immediato in caso di difficoltà.

Quotidianamente era prevista una breve riunione di allineamento per monitorare i progressi e affrontare le eventuali problematiche emerse. Infine, ogni venerdì mattina si teneva una sessione di revisione più formale, dedicata alla presentazione dei risultati settimanali, all'analisi della direzione del progetto e alla discussione di eventuali proposte o azioni correttive.

2.2. Pianificazione delle attività

L'organizzazione delle attività è stata calcolata in divenire, in modo da allocare correttamente le risorse disponibili e avere sotto controllo il raggiungimento degli obiettivi, restando entro il tempo prestabilito dal tirocinio. In autonomia è stato stilato un calendario settimanale su Notion_G, in cui veniva tenuta traccia degli obiettivi giornalieri. Questo strumento ha permesso di monitorare efficacemente lo stato di avanzamento, offrendo la flessibilità di riprogrammare i task rivelatisi più complessi o di anticipare le attività successive in caso di completamento anticipato del carico giornaliero.

Questo crono-programma veniva poi condiviso a fine settimana per mostrare il progresso raggiunto e condividere le attività che si erano rivelate più onerose, al fine di chiedere un consiglio ai colleghi più esperti per le problematiche ancora aperte.

2.2.1. Prima settimana

L'attività di tirocinio si è aperta con una fase di inserimento in azienda, focalizzata sull'acquisizione del metodo di lavoro e sulla strutturazione delle attività di sviluppo del progetto. In seguito è stato fatto uno studio approfondito delle principali tecnologie con le quali si sarebbe sviluppata la piattaforma, partendo dal framework di Angular_G e dal suo linguaggio TypeScript_G. Lo studio teorico dei concetti è stato affiancato da esercizi, tutorial e progetti didattici orientati alla comprensione dei moduli principali del framework, facendo sempre riferimento alla documentazione ufficiale [2].

2.2.2. Seconda settimana

Durante la seconda settimana è continuato lo studio di Angular_G, vedendo concetti più avanzati come il *routing*, gli *observer*, la *dependency injection*. In seguito è stato affrontato il dominio del problema, studiando i concetti di CVE_G e di CPE_G. Questa analisi è stata integrata dalla lettura di articoli scientifici inerenti alle tematiche del Vulnerability Assessment_G e della prioritizzazione del rischio.

2.2.3. Terza settimana

In questo periodo è stato completato lo studio del dominio ed è iniziata l'analisi dei requisiti della piattaforma, delineando le principali funzionalità dell'applicazione e la sua interfaccia. Contestualmente è stata fatta formazione riguardo alla piattaforma di Qualys_G, prodotto enterprise utilizzato

dall'azienda per scansionare le vulnerabilità di dispositivi. È stata poi fatta della formazione sull'utilizzo di Docker e Docker Compose, raffinando i concetti già presenti per avere un più consapevole utilizzo dello strumento.

2.2.4. Quarta settimana

Durante la quarta settimana è iniziata la progettazione del sistema, definendo le classi di backend_G e le componenti Angular_G con le loro principali funzioni. Per formalizzare ed esporre i risultati del lavoro sono stati prodotti dei diagrammi in UML_G, per discutere della direzione progettuale e raffinare il lavoro compiuto. In aggiunta è stata progettata l'organizzazione della repo e dei suoi file, in modo da poter procedere con uno sviluppo il più possibile ordinato ed efficiente. È continuato lo studio delle tecnologie al fine di comprendere i pattern di organizzazione del codice di Angular_G e FastAPI_G, in modo da utilizzarle nello sviluppo della piattaforma rispettando la progettazione architetturale.

2.2.5. Quinta settimana

In questo periodo è iniziata la codifica del prodotto, partendo dalle funzionalità di backend_G. Si è sviluppato in modo progressivo integrando man mano le parti che contribuivano al sistema di calcolo delle priorità. Questa modularità ha facilitato la scrittura dei test e ha consentito di produrre risultati parziali ma concreti durante tutto il ciclo di implementazione. In particolare sono stati raggiunti questi obiettivi:

- codifica della struttura di API_G di backend_G, dalla quale si interfacerà il frontend_G
- codifica del *service* principale che orchestra la pipeline di analisi
- impostazione di interfaccia minimale per l'avvio di analisi dal web
- integrazione degli adapter per il recupero di dati reali per l'analisi (KEV_G, EPSS_G, CVSS_G)

Per accelerare la codifica di questa parte e testare la validità degli studi fatti riguardo al dominio è stato fatto uso di *fixture* di dati (risposte sintetiche delle chiamate API_G dei vari servizi), in modo da strutturare la logica di backend_G anche senza avere le credenziali di Qualys_G o Claude_G.

2.2.6. Sesta settimana

Nella sesta settimana è stato integrato nel sistema l'AI_G, tuttavia è stato necessario virare ad un modello gratuito come *Gemini-Flash*, per una mancanza di credenziali di un modello a pagamento come *Claude Sonnet*. L'utilizzo di un modello gratuito è stato fortemente limitante per un'analisi accurata di tutte le CVE_G trovate dallo scanner. Per questo motivo nell'MVP_G dimostrativo è stato necessario far generare una spiegazione solo alle cinque più gravi vulnerabilità del dispositivo target. In seguito è stato anche impostato l'ambiente di test con la macchina virtuale di Qualys_G con utenza al tenant_G cloud_G per interfacciarsi alle API_G di lancio analisi e recupero dati. Per la codifica dell'adapter di Qualys_G è stato necessario affrontare un accurato studio della sua documentazione API_G [3].

2.2.7. Settima settimana

Durante questa settimana è stato raffinato il recupero di informazioni tramite API_G, risolvendo un problema di limiti di richieste e numero di dati richiesti. In seguito è stato completato e stilizzato il frontend, pensato per essere piacevole e funzionale ad una figura come un analista di sicurezza; più precisamente, sono state ultimate:

- pagina di home con lista di scansioni effettuate.
- pagina di lancio scansione, con pipeline_G visiva per monitorare l'avanzamento del processo.
- pagina di report con una dettagliata spiegazione di come interpretare i dati forniti e una tabella dimostrativa e piacevole da consultare con funzionalità di ordinamento per campo dati selezionato.
- bottone di esportazione con annessa funzionalità di creazione report in formato DOCX_G.

2.2.8. Ottava settimana

Durante questa fase, è stato progettato e implementato un sistema di gestione delle eccezioni personalizzato e granulare, al fine di segnalare in modo preciso e completo all'utente eventuali

fallimenti durante la generazione del report delle vulnerabilità. Nello specifico sono state modellate delle classi di errore dedicate, sollevate in risposta a casi di errore di API_G esterne. Queste eccezioni vengono poi normalizzate in una struttura standardizzata, comprensiva di codice errore, descrizione ed altri parametri di contesto. Questo approccio serve sia per notificare correttamente l'utente e migliorare la *user experience*, sia per agevolare lo sviluppatore in fase di *debugging_G* della piattaforma.

2.2.9. Nona settimana

Nell'ultima settimana è stato raffinato il lavoro compiuto, procedendo con un'attenta fase di *refactoring_G* del codice. Questo intervento ha permesso di ordinare e modularizzare le funzioni presenti nella codebase_G, garantendo un codice più pulito, coerente e facilmente manutenibile in vista di sviluppi futuri. È stato infine predisposto un sito web statico per la documentazione del codice, in modo da lasciare una guida di riferimento che può essere mantenuta per agevolare l'orientamento e l'intervento di futuri sviluppatori.

Capitolo 3.

Studio del Dominio

Durante la prima fase del tirocinio è stato fatto uno studio approfondito sul tema del Vulnerability Assessment_G, attraverso il quale sono stati compresi i termini chiave del dominio, le principali problematiche e le soluzioni che le aziende prendono in considerazione per gestire al meglio il tracciamento e la risoluzione di vulnerabilità. Come riporta Rajamani [4], nel solo 2025 sono state pubblicate 48.185 CVE_G, di cui il 56% è classificato come *high* o *critical*, rendendo la coda di remediation_G ingestibile senza un triage_G intelligente. Questa eccessiva segnalazione di vulnerabilità porta ad un fenomeno chiamato Vulnerability Fatigue / Alert Fatigue_G che induce gli analisti a non svolgere in modo efficace la loro gestione. Per questo motivo occorre adottare un framework CTEM_G (*Continuous Threat Exposure Management*) e affidarsi ad un ampio ventaglio di metriche per poter contestualizzare al meglio le CVE_G ed ottimizzare la loro risoluzione, dando priorità ad un ristretto e mirato numero di vulnerabilità.

La piattaforma di ThreatLens si sviluppa in questo contesto e intende ottenere da un indirizzo IP_G una lista di CVE_G utilizzando scanner professionali di terze parti come Qualys_G. Ai risultati ritornati dallo strumento aggiunge una serie di altre metriche, fornendo un dettagliato e attendibile contesto facilmente interpretabile da un analista di sicurezza. La piattaforma funge da motore di prioritizzazione intelligente: valuta il rischio operativo di ogni singola CVE_G e impiega l'Intelligenza Artificiale per generare una spiegazione chiara e sintetica delle metriche rilevate. In questo modo, il sistema non si sostituisce all'analista nella decisione finale, ma gli fornisce un quadro contestualizzato e argomentato, permettendogli di validare rapidamente le informazioni proposte e procedere in modo tempestivo con la corretta remediation_G.

3.1. Limiti del CVSS

La metrica principale per classificare una CVE_G è il CVSS_G, uno standard sviluppato dal FIRST_G che indica da 1.0 a 10.0 la gravità di una vulnerabilità. Questa metrica, tuttavia, risponde alla domanda sbagliata per il triage_G: dice quanto sarebbe grave una vulnerabilità se sfruttata, non quanto è probabile che venga sfruttata in tempi utili per decidere il patching_G. Come evidenziato da Shimizu e Hashimoto [1], la metrica CVSS_G è stata creata solo per misurare il massimo impatto teorico possibile invece del rischio nel mondo reale. Infatti non considera la probabilità che essa venga usata e il rischio pesato in un preciso contesto. Già da diverso tempo, numerosi studi in letteratura [5] sottolineano come i punteggi CVSS_G, senza informazioni sull'ambiente in cui si presenta la vulnerabilità, abbiano utilità limitata per una prioritizzazione pratica. Inoltre, aggiungere del contesto migliorerebbe significativamente la selezione delle risposte.

Un secondo problema di questa metrica è la scarsa azionabilità del punteggio. Il CVSS_G concentra molte vulnerabilità nelle fasce alte, con uno score superiore al 7.0, rendendo difficile stabilire la priorità di remediation e aumentando il fenomeno della Vulnerability Fatigue / Alert Fatigue_G.

3.2. Nuove metriche supportate dalla letteratura scientifica

La soluzione sarebbe integrare nella prioritizzazione una serie di altre metriche per sopperire ai problemi sopra descritti. Un dato significativo è l'EPSS_G, che utilizza il machine learning per stimare la probabilità che una vulnerabilità venga sfruttata entro 30 giorni, mentre il KEV_G indica se sono state registrate evidenze di un exploit_G confermato. Secondo Shimizu e Hashimoto [1], mentre un filtro basato esclusivamente sul CVSS_G presenta un'efficienza operativa di appena lo 0,5% (generando un elevato rumore di fondo), il modello combinato innalza la precisione al 9,1%.

Questo permette di concentrare gli sforzi operativi sulle minacce reali, mantenendo al contempo una copertura (*coverage*) dell'85,6% sulle vulnerabilità effettivamente sfruttate.

Tabella 1: Confronto delle metriche di prioritizzazione: Efficienza e Copertura in base al metodo adottato [1].

Method	Efficiency	Coverage
$CVSS \geq 7.0$	0.5%	90.0%
KEV Only	74.3%	86.7%
$EPSS \geq 0.088$	4.9%	48.9%
Proposed Method $(KEV \vee EPSS) \wedge CVSS$	9.1%	85.6%

3.3. Il modello Vulnerability Management Chaining

Shimizu e Hashimoto [1] dimostrano come combinare $CVSS_G$, $EPSS_G$ e KEV_G dia un risultato molto più accurato. Lo studio definisce il *Vulnerability Management Chaining*, un albero decisionale (*decision tree*) che prende in input questi tre dati e restituisce una priorità da assegnare alla CVE_G .

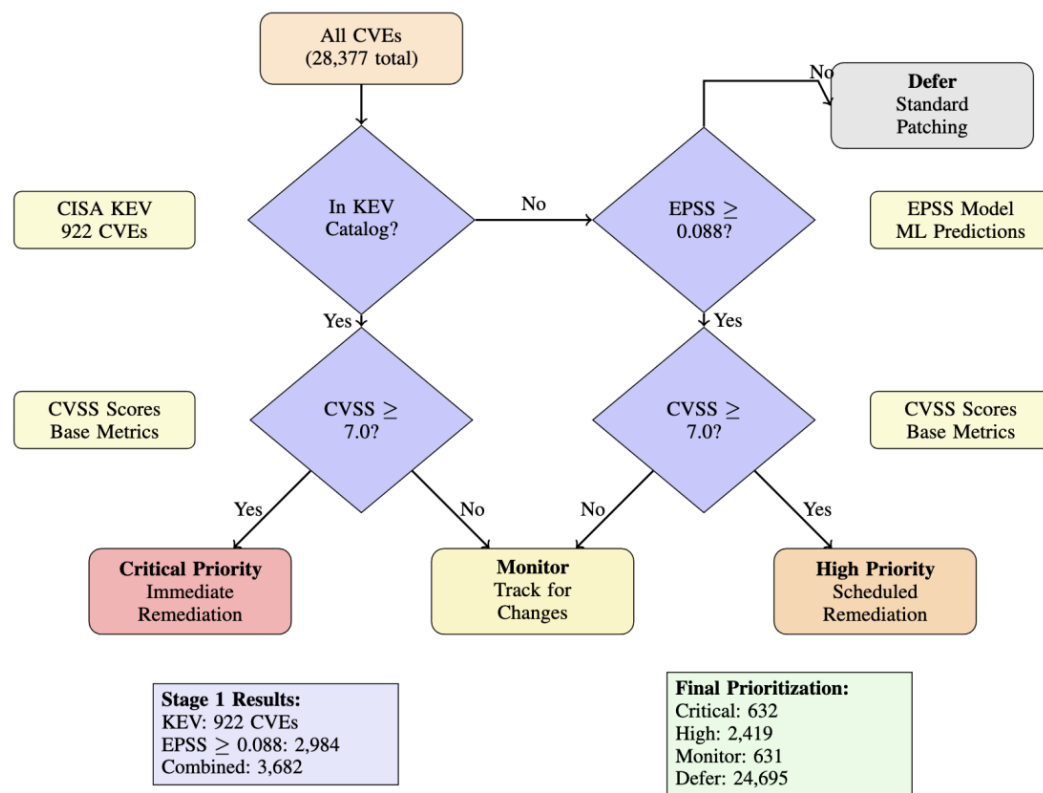


Figura 2: *Vulnerability Management Chaining Decision Tree*, da [1]

L'albero presentato in Figura 2 è composto da due stadi principali:

1. Nel primo stadio viene controllata la reale minaccia e si passa allo stadio successivo se e solo se una di queste due condizioni è vera:
 - la vulnerabilità è presente nel catalogo del KEV_G ?
 - presenta un $EPSS_G$ alto (maggiore di 0.0888)?

2. Se si arriva al secondo stadio, la minaccia è probabile, dunque si valuta quanto sarebbe grave nel peggior caso possibile: se la vulnerabilità presenta un $CVSS_G$ maggiore di 7.0 è di priorità massima.

Dal *decision tree_G* possono essere prodotti in output quattro risultati:

Tabella 2: Regole decisionali e condizioni logiche per l'assegnazione delle classi di priorità.

Classe di Priorità	Condizione Logica (Decision Tree)
Critical	$KEV == \text{TRUE} \text{ AND } CVSS \geq 7.0$
High	$EPSS \geq 0.0888 \text{ AND } CVSS \geq 7.0 \text{ AND } KEV == \text{FALSE}$
Monitor	$(KEV == \text{TRUE} \text{ OR } EPSS \geq 0.0888) \text{ AND } CVSS < 7.0$
Defer	$KEV == \text{FALSE} \text{ AND } EPSS < 0.0888$

Nella piattaforma ThreatLens, per la classificazione dell'esito finale, il modello $SSVC_G$ viene impiegato come linguaggio di riferimento.

Questa scelta progettuale è motivata dal fatto che il modello $SSVC_G$ viene adottato come linguaggio di classificazione, in quanto progettato esplicitamente per categorizzare le decisioni di risposta attorno agli *stakeholder_G*, alle azioni di mitigazione e alla tolleranza al rischio dell'organizzazione. Come riporta la documentazione ufficiale di CISA [6]:

CISA uses its own SSVC decision tree model to prioritize relevant vulnerabilities into four possible decisions:

- **Track:** *The vulnerability does not require action at this time. The organization would continue to track the vulnerability and reassess it if new information becomes available. CISA recommends remediating Track vulnerabilities within standard update timelines.*
- **Track*:** *The vulnerability contains specific characteristics that may require closer monitoring for changes. CISA recommends remediating Track* vulnerabilities within standard update timelines.*
- **Attend:** *The vulnerability requires attention from the organization's internal, supervisory-level individuals. Necessary actions include requesting assistance or information about the vulnerability, and may involve publishing a notification either internally and/or externally. CISA recommends remediating Attend vulnerabilities sooner than standard update timelines.*
- **Act:** *The vulnerability requires attention from the organization's internal, supervisory-level and leadership-level individuals. Necessary actions include requesting assistance or information about the vulnerability, as well as publishing a notification either internally and/or externally. Typically, internal groups would meet to determine the overall response and then execute agreed upon actions. CISA recommends remediating Act vulnerabilities as soon as possible.*

Di conseguenza, gli esiti dell'elaborazione algoritmica vengono mappati sulle seguenti categorie d'azione standardizzate:

- TRACK
- TRACK*
- ATTEND
- ACT

3.3.1. Analisi teorica del funzionamento

I tre dati da soli forniscono poche informazioni, ma combinati si compensano tra di loro, dando un contesto più completo:

- il KEV_G ha un'alta confidenza (se presente nel database è un dato molto rilevante), ma ha una copertura limitata e una natura reattiva (non prevede rischi potenzialmente gravi);

- l'EPSS_G ha una forte copertura predittiva, ma proprio per questo è un dato probabilistico con alta incertezza che produce falsi positivi e falsi negativi;
- il CVSS_G valuta l'impatto potenziale in modo accurato, ma non indica se la vulnerabilità sarà realmente sfruttata.

Questo framework adotta un approccio *threat-first*: come riportano Shimizu e Hashimoto [1], le vulnerabilità realmente sfruttate rispetto alle CVE_G pubblicate sono in numero molto minore. Per questo motivo, l'algoritmo parte valutando le vulnerabilità che sono realmente una minaccia.

3.3.2. Parametrizzazione dei valori

Le soglie citate sopra per CVSS_G e EPSS_G sono dei parametri indicati da Shimizu e Hashimoto [1] che hanno condotto un'analisi usando un dataset di 28.377 CVE_G. Lo studio dimostra come questi specifici valori siano statisticamente ottimali per segmentare le minacce in classi operative distinte.

Tuttavia, lo studio indica che questi valori possono essere dei parametri flessibili ed è possibile configurarli in funzione del profilo di tolleranza che si vuole ottenere. Se, per esempio, ci si trova in un ambiente particolarmente critico ed è necessario prendere in considerazione anche delle vulnerabilità con valori di gravità minore, si abbassa la soglia EPSS_G. Se un'organizzazione ha meno risorse da allocare per il Vulnerability Assessment_G, si può decidere di alzare la soglia accettando un rischio maggiore, al fine di isolare un quantitativo minore di vulnerabilità da gestire.

In ThreatLens è stato deciso di inserire nell'algoritmo le soglie indicate come statisticamente ottimali da Shimizu e Hashimoto [1]. Tuttavia, a fronte delle considerazioni sopracitate, è stata presa in considerazione come futuro sviluppo una configurazione dall'interfaccia web del calcolo della priorità. Questa implementazione consentirà all'analista di selezionare queste soglie calibrando l'algoritmo per adattare i risultati allo specifico contesto operativo e alla tolleranza al rischio della propria organizzazione.

3.3.3. Ruolo del contesto dell'asset

Il sistema progettato analizza il contesto del dispositivo da analizzare attraverso tre parametri distinti forniti in input dall'utente: l'ambiente di esecuzione, l'esposizione di rete e la criticità sistemica dell'asset. Il sistema non utilizza questi dati per alterare matematicamente la priorità finale della vulnerabilità, ma li elabora per produrre un indicatore parallelo e indipendente che segnala all'analista se il contesto richiede un'attenzione nulla, elevata o forte. Il calcolo è rappresentato dalla matrice Tabella 3:

Tabella 3: Matrice di calcolo deterministico per il livello di attenzione contestuale (*Context Attention*).

Criticità Esposizione	LOW	MEDIUM	HIGH
ISOLATED	NONE	NONE	ELEVATED
INTERNAL	NONE	ELEVATED	STRONG
PUBLIC	ELEVATED	STRONG	STRONG

Questa scelta progettuale è motivata dallo stato attuale della ricerca scientifica al riguardo. Sebbene la letteratura [7] riconosca che il contesto dell'asset sia un livello informativo cruciale per la prioritizzazione, non esistono ad oggi criteri empiricamente consolidati per convertire tali fattori in pesi numerici esatti.

Per garantire un rigore scientifico ed evitare di generare metriche non validate, si è deciso di trattare il contesto dell'asset come un segnale separato che arricchisce l'analisi generale. In questo modo si evita di inquinare il calcolo della gravità modellato su algoritmi comprovati dalla letteratura del settore [1].

Capitolo 4.

Analisi dei Requisiti

A seguito dello studio del dominio del problema, si è proceduto alla definizione dei casi d'uso e alla conseguente stesura dei requisiti di sistema.

4.1. Casi d'uso del sistema

UC1: Creazione e avvio dell'analisi di sicurezza

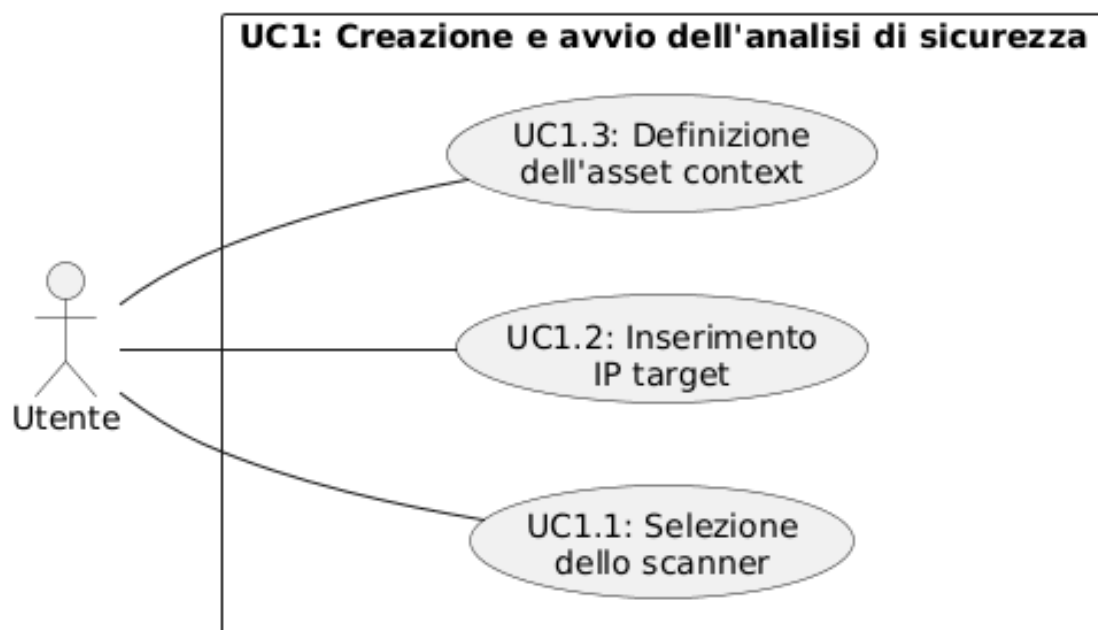


Figura 3: Use Case - UC1: Creazione e avvio dell'analisi di sicurezza

Attore principale	utente
Attore secondario	Scanner
Precondizione	L'utente si trova all'interno della piattaforma nella sezione di nuova analisi.
Flusso principale	L'utente configura i parametri fondamentali per la scansione (scanner, target, contesto) e ne richiede l'avvio. Il sistema convalida i dati in ingresso e avvia l'analisi.
Sottocasi inclusi	UC1.1 (Selezione scanner), UC1.2 (Inserimento IP), UC1.3 (Definizione Asset Context).
Postcondizione	La scansione è avviata correttamente e il sistema entra in fase di elaborazione.

UC1.1: Selezione dello scanner

Attore principale utente

Flusso principale L'utente seleziona lo scanner da utilizzare per l'analisi. Attualmente il sistema vincola la selezione all'unica opzione supportata (Qualys_G).

UC1.2: Inserimento IP_G target

Attore principale utente

Flusso principale L'utente inserisce un singolo indirizzo IP_G che rappresenta il target su cui effettuare l'analisi delle vulnerabilità.

UC1.3: Definizione dell'asset context_G

Attore principale utente

Flusso principale L'utente seleziona i tre parametri di contesto necessari a delineare il profilo di rischio del target: ambiente (*environment*), esposizione (*exposure*) e criticità (*criticality*).

UC2: Monitoraggio dell'avanzamento dell'analisi



Figura 4: Use Case - UC2: Monitoraggio dell'avanzamento dell'analisi

Attore principale utente

Flusso principale L'utente visualizza l'interfaccia dedicata allo stato dell'analisi. Il sistema interroga lo scanner e mostra in tempo reale l'avanzamento del processo (scansione in corso, recupero di dati, generazione AI_G o pipeline_G fallita).

Postcondizione L'utente è costantemente informato sullo stato di completamento del task.

UC3: Apertura e consultazione del report di vulnerabilità

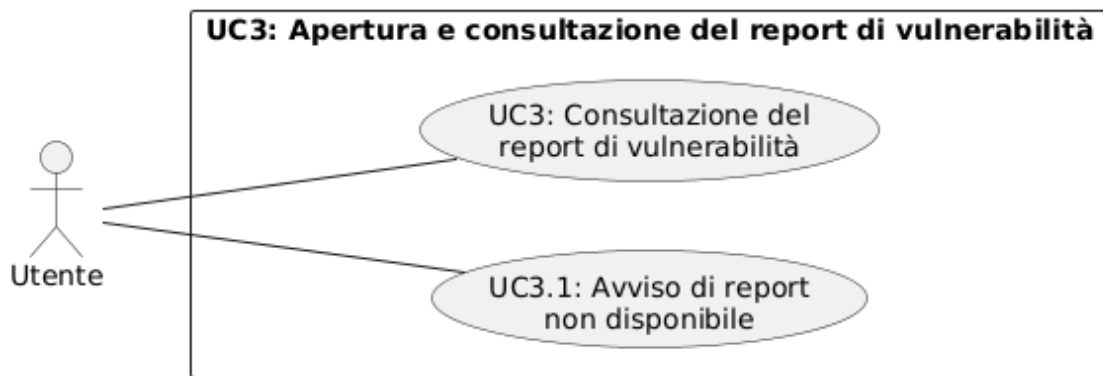


Figura 5: Use Case - UC3: Apertura e consultazione del report di vulnerabilità

Attore principale	utente
Precondizione	L'utente richiede l'accesso ai risultati di una specifica analisi.
Flusso principale	Il sistema recupera il report e presenta un <i>Vulnerability Report</i> aggregato che include le vulnerabilità, la loro priorità operativa e la spiegazione generata dall'AI _G .
Sottocasi inclusi	UC3.1 (Report non disponibile)
Postcondizione	L'utente dispone delle metriche contestuali per prendere una decisione operativa sulla remediation _G .

UC3.1: Report non disponibile

Attore principale	utente
Flusso principale	Qualora l'analisi non sia ancora terminata, sia fallita durante la pipeline _G o il report richiesto non esista nel sistema, viene mostrato un avviso a schermo che comunica esplicitamente che il report non è disponibile.
Postcondizione	L'utente è informato dell'indisponibilità del dato.

UC4: Esportazione del report

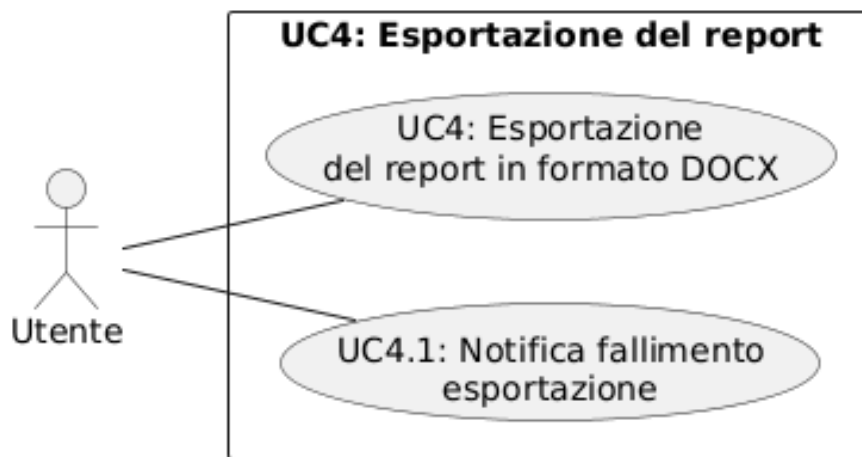


Figura 6: Use Case - UC4: Esportazione del report

Attore principale	utente
Precondizione	L'utente sta visualizzando un report completato e accessibile.
Flusso principale	L'utente richiede l'esportazione del report. Il sistema compila un documento in formato DOCX _G contenente il dettaglio tecnico delle vulnerabilità e lo rende disponibile per il download.
Sottocasi inclusi	UC4.1 (Fallimento esportazione)
Postcondizione	Viene scaricato il file DOCX _G del report.

UC4.1: Fallimento esportazione del report

Attore principale	utente
Flusso principale	Se il processo di compilazione del file DOCX _G o il suo salvataggio incontra un'eccezione, il sistema interrompe il processo di esportazione e notifica l'errore all'utente tramite un apposito messaggio.
Postcondizione	Il sistema segnala il fallimento dell'operazione e l'esportazione viene annullata.

UC5: Invio del report tramite email

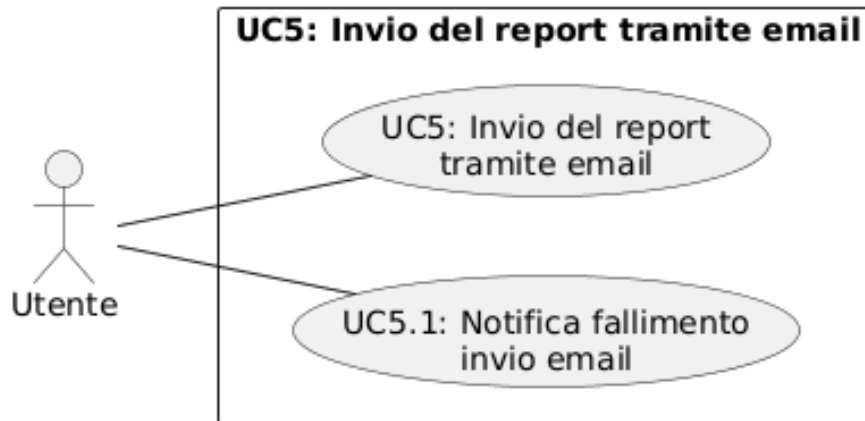


Figura 7: Use Case - UC5: Invio del report tramite email

Attore principale	utente
Flusso principale	L'utente, dopo aver visualizzato un'analisi completata, richiede l'invio del report tramite email e specifica l'indirizzo di destinazione. Il sistema predispone il documento (es. in formato DOCX _G), lo allega a un messaggio e lo inoltra al server SMTP per la consegna.
Postcondizione	Il report viene inviato con successo all'indirizzo specificato e il sistema conferma all'utente la presa in carico dell'operazione.

UC5.1: Fallimento invio del report tramite email

Attore principale	utente
Flusso principale	Se il sistema rileva un errore durante la comunicazione con il server di posta (ad esempio per problemi di rete, timeout o credenziali non valide) o se l'indirizzo email fornito risulta malformato, il processo di invio viene interrotto.
Postcondizione	L'email non viene inoltrata e il sistema notifica l'errore all'utente, invitandolo a riprovare o a controllare i dati inseriti.

4.2. Requisiti Funzionali

Nella seguente tabella sono riportati i requisiti funzionali obbligatori estratti dai casi d'uso.

Tabella 4: Elenco dei Requisiti Funzionali obbligatori estratti dai casi d'uso.

Codice	Descrizione	Fonti	Stato
RF001	Il sistema deve permettere all'utente di richiedere l'avvio di una nuova analisi.	UC1	Implementato
RF002	L'utente deve poter selezionare lo scanner dall'interfaccia. Il sistema deve forzare l'unica opzione supportata (Qualys _G).	UC1.1	Implementato
RF003	L'utente deve poter inserire un singolo indirizzo IP _G in un apposito campo di testo.	UC1.2	Implementato
RF004	L'utente deve poter selezionare i parametri dell'asset context _G (ambiente, esposizione e criticità) attraverso appositi menu a tendina o selettori.	UC1.3	Implementato
RF005	Il sistema deve mostrare all'utente un indicatore visivo in tempo reale con lo stato esatto della scansione (es. in corso, recupero dati, AI, fallita).	UC2	Implementato
RF006	Il sistema deve renderizzare a schermo il Vulnerability Report, mostrando le vulnerabilità, il badge della priorità operativa calcolata e il testo dell'AI _G .	UC3	Implementato
RF007	Il sistema deve mostrare all'utente un avviso di Report non disponibile se si tenta di aprire un'analisi inesistente, ancora in esecuzione o fallita.	UC3.1	Implementato
RF008	L'utente deve poter cliccare un comando specifico per richiedere la compilazione e il download del report in formato DOCX _G .	UC4	Implementato
RF009	Il sistema deve intercettare gli errori di generazione file e avvisare l'utente con un messaggio d'errore a schermo.	UC4.1	Implementato
RF010	L'utente deve poter inviare il report finale ad un indirizzo email specifico.	UC5	Non Implementato.
RF011	Il sistema deve intercettare errori nell'invio tramite email del report.	UC5.1	Non Implementato.

Capitolo 5.

Tecnologie

5.1. Tecnologie di Backend

Per lo sviluppo del backend_G si è adottato un approccio moderno basato su Python. Di seguito sono riportate le tecnologie e le dipendenze che compongono l'infrastruttura lato server.

Tabella 5: Tecnologie di Backend

Nome	Versione	Descrizione
Linguaggio e Containerizzazione		
Python	3.12	Linguaggio di programmazione utilizzato per il backend _G , scelto per il suo ampio numero di librerie, la velocità di sviluppo e poiché è il linguaggio di FastAPI, uno dei principali framework di backend _G .
Docker & Docker Compose	-	Piattaforma di containerizzazione utilizzata per l'isolamento dell'ambiente di esecuzione e l'orchestrazione dei servizi collegati.
Infrastruttura Web (API & Server)		
FastAPI	0.138.2	Framework web ad alte prestazioni per la costruzione di API RESTful. Gestisce il routing, il middleware (es. CORS) e la gestione degli errori HTTP.
Uvicorn	0.49.0	Server web ASGI leggero e veloce, responsabile dell'esecuzione asincrona dell'applicazione FastAPI.
Validazione e Configurazione		
Pydantic	2.13.4	Libreria per la validazione rigida e type-safe. Viene utilizzata per validare tutti i dati in entrata e in uscita del backend _G , supportata nativamente da FastAPI.
Pydantic Settings	2.14.2	Estensione di Pydantic impiegata per la gestione centralizzata e sicura delle configurazioni e delle variabili d'ambiente come le API key.
Integrazioni e Client HTTP		
HTTPX	0.28.1	Client HTTP di nuova generazione, impiegato per le chiamate di rete verso i provider esterni.
Requests	-	Libreria HTTP standard utilizzata in modalità sincrona all'interno degli adapter per interrogare gli scanner e le API esterne (es. Qualys, NVD).
Google GenAI	-	SDK _G ufficiale integrato nell'adapter AI _G per orchestrare le chiamate al modello LLM responsabile della generazione delle spiegazioni.
Generazione Documentale		
Python-docx	1.2.0	Libreria impiegata per la formattazione del report finale di vulnerabilità in formato DOCX _G .

Strumenti di Sviluppo (Dev Tools)		
Ruff	0.15.20	Lint e formatter ad alte prestazioni scritto in Rust, utilizzato per garantire la pulizia e lo standard stilistico del codice sorgente.
Pytest	9.1.1	Framework per la stesura e l'esecuzione dei test automatici.

5.2. Tecnologie di Frontend

Per lo sviluppo del frontend è stato scelto il framework Angular 22, per il suo vasto ecosistema di programmazione web (che include il routing nativo), per una forte scalabilità per la sua architettura a componenti, per la presenza di pattern integrati nel sistema come l'iniezione delle dipendenze, e per l'uso di moderne librerie come RxJS per la programmazione reattiva.

Tabella 6: Tecnologie di frontend

Nome	Versione	Descrizione
Linguaggi di Programmazione e Markup		
TypeScript	6.0.2	Linguaggio utilizzato per il frontend che garantisce un type-checking rigoroso, nativo del framework di Angular.
HTML5 & CSS3	Nativi	Linguaggi standard utilizzati per la strutturazione semantica della pagina e per il design dei componenti utente.
Framework e Moduli Core		
Angular	^22.0.0	Framework open source per lo sviluppo di Single-Page Application. È stato usato per realizzare il frontend dell'applicazione.
RxJS	7.8.0	Libreria per la programmazione reattiva basata su Observable , fondamentale per la gestione dei flussi asincroni e degli eventi nel framework Angular.

5.3. Strumenti di Sviluppo e Ambiente

Durante il ciclo di vita del software è stata inoltre usata una serie di strumenti trasversali che hanno garantito un corretto sviluppo della piattaforma.

Tabella 7: Strumenti di sviluppo e ambiente

Nome	Versione	Descrizione
Codifica e Versionamento		
Visual Studio Code	-	Editor di codice sorgente avanzato utilizzato come ambiente di sviluppo integrato (IDE) principale, configurato con estensioni per il supporto nativo a Python, TypeScript e Typst.
Git	-	Sistema di controllo di versione distribuito, impiegato per il tracciamento progressivo delle modifiche, la gestione dei <i>branch</i> e la storicizzazione sicura del progetto.
Ambiente di Esecuzione e Test		
VMware	-	Software <i>hypervisor</i> utilizzato per l'esecuzione locale di macchine virtuali isolate. È risultato fondamentale per istanziare i target di test da analizzare tramite lo scanner.
Qualys Virtual Scanner	-	Appliance virtuale fornita da Qualys, eseguita in locale per materializzare le scansioni sui target interni comunicando con l'infrastruttura Cloud.
Metasploitable 2	-	Macchina virtuale basata su Linux, intenzionalmente vulnerabile. È stata impiegata come ambiente <i>target</i> controllato per validare l'effettivo rilevamento delle minacce.
Servizi Esterni e API		
Qualys Tenant API	-	Interfaccia REST esposta dal Cloud Qualys per orchestrare il processo di scansione, inviare comandi all'appliance locale ed estrarre i referti tecnici.
Google AI API	-	Servizio di intelligenza artificiale interrogato per generare la spiegazione contestuale delle vulnerabilità rilevate, sfruttando avanzati modelli linguistici.

NVD API	-	API del National Vulnerability Database interrogata per recuperare le metriche base e il punteggio di severità (CVSS _G) associato alle singole vulnerabilità.
CISA KEV Catalog	-	Catalogo fornito dalla CISA e interrogato tramite feed JSON per verificare in modo deterministico se una specifica vulnerabilità risulta attivamente sfruttata (<i>Exploited</i>).
FIRST EPSS API	-	Endpoint REST fornito dal framework FIRST per il recupero dell'EPSS _G , utilizzato per stimare la probabilità di sfruttamento di una falla entro 30 giorni.
Progettazione, Appunti e Documentazione		
PlantUML	-	Strumento utilizzato per la modellazione e la generazione automatica di diagrammi architetturali.
Notion	-	Piattaforma di produttività impiegata come <i>knowledge base</i> centralizzata per la raccolta degli appunti, la stesura dei requisiti e il tracciamento delle decisioni progettuali.
MkDocs	-	Generatore di siti web statici tipicamente usato per documentare il codice di progetti Python tramite file in formato Markdown.

Capitolo 6.

Ambiente di test

6.1. Architettura del laboratorio

L'ambiente di collaudo di ThreatLens è costituito da due macchine virtuali (VM) che comunicano tra di loro tramite un'interfaccia di rete in modalità **bridge**. La prima VM ospita uno scanner Qualys_G incaricato di eseguire le scansioni di vulnerabilità sulla macchina target e di inviare i risultati al tenant_G cloud di Qualys_G. ThreatLens utilizza le API proprietarie di Qualys_G per inviare il segnale di inizio scansione alla VM scanner e poi, sempre tramite API, rileva i risultati delle scansioni. La seconda VM ospita Metasploitable 2, una macchina Linux intenzionalmente vulnerabile, usata per condurre test per strumenti di sicurezza e praticare tecniche di *pentesting_G* comuni.

6.2. Problemi e soluzioni adottate

A causa dell'indisponibilità temporanea del datacenter aziendale di Kirey, l'infrastruttura di virtualizzazione è stata ospitata localmente sulla workstation personale fornita dall'azienda. Tale vincolo ha imposto un'allocazione stringente delle risorse hardware: 2 GB di RAM e 1 vCPU per l'istanza Metasploitable, e 4 GB di RAM e 2 vCPU per l'appliance Qualys_G.

Questo ridimensionamento ha inizialmente influito negativamente sui cicli di sviluppo, portando i tempi di esecuzione di una singola scansione fino a superare un'ora e mezza. Per ottimizzare il processo di sviluppo e testing della piattaforma, è stata sfruttata la modularità architetturale della pipeline_G di elaborazione. Tramite un branch di Git dedicato è stata saltata la fase di scansione *real-time* ed è stata eseguita l'elaborazione dei dati da una scansione precedentemente lanciata.

Questo approccio ha permesso di snellire lo sviluppo senza compromettere l'affidabilità della piattaforma. Infatti, il meccanismo di avvio della scansione era già stato testato e validato, mentre il fulcro della complessità logica risiedeva nel recupero dei dati e nel calcolo della priorità delle CVE_G.

Capitolo 7.

Progettazione Architettuale

L'obiettivo della fase di progettazione è stato delineare la struttura di un sistema in grado di rispettare i requisiti derivati dalla fase di studio del dominio. La fase iniziale è stata dedicata alla costruzione del backend_G, costruendo i diagrammi delle classi per modellare le principali entità del sistema. A questa fase è stata data particolare attenzione poiché è il backend_G che contiene tutta la logica del sistema, il recupero dei dati e il motore di classificazione delle vulnerabilità, delegando al frontend_G solo la parte di interfaccia. Successivamente è stato definito il frontend_G, ricercando quali fossero i pattern più usati nel framework di Angular per strutturare correttamente un'interfaccia web.

7.1. Architettura di backend_G: Ports & Adapters

Il backend_G è stato modellato con un'architettura esagonale, conosciuta anche come *Ports & Adapters*. Questo tipo di struttura ha come obiettivo primario isolare la logica di business, garantendo un'elevata testabilità e indipendenza da tecnologie esterne. Il *core_G* in questo modo risulta completamente agnostico rispetto ai dettagli implementativi di framework, database o provider di dati esterni. Il sistema è strutturato in livelli concentrici:

- *Domain*: rappresenta il nucleo dell'architettura, contiene le entità (modellate tramite classi) sulle quali si basa tutto il sistema ed è privo di dipendenze esterne;
- *Services*: definiscono i casi d'uso dell'applicazione e fungono da orchestratori. Hanno il compito di implementare le *Inbound Ports* per gestire le richieste in ingresso, coordinano gli oggetti del dominio e utilizzano le *Outbound Ports* per delegare all'esterno operazioni come salvataggio o recupero di dati;
- *Ports*: sono il punto di connessione tra il dominio e l'esterno, permettendo una comunicazione strutturata senza creare accoppiamento. Si suddividono in *Inbound Ports* (definiscono i casi d'uso accessibili dall'esterno) e *Outbound Ports* (modellano interfacce per dialogare con l'esterno);
- *Adapters*: rappresentano lo strato più esterno e operano come traduttori tra le tecnologie specifiche e il nucleo. Si suddividono in *Inbound Adapters* (guidano l'input invocando le *Inbound Ports*) e *Outbound Adapters* (vengono guidati dall'applicazione per interagire con l'infrastruttura esterna tramite le *Outbound Ports*).

7.1.1. Diagramma delle classi di backend_G

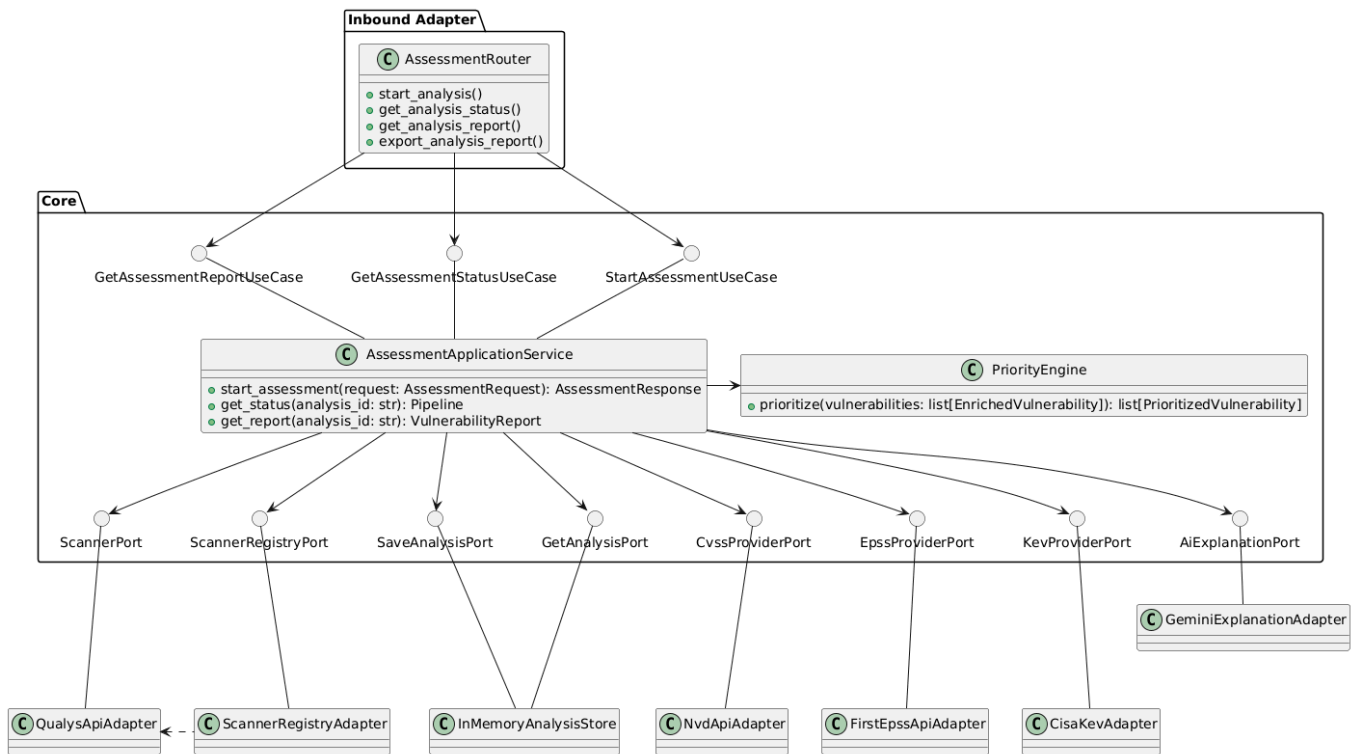


Figura 8: Diagramma architetturale dei componenti

Il diagramma in Figura 8 modella le principali componenti di backend_G.

7.1.2. Inbound Adapters

L'unico *Inbound Adapter* è l'**AssessmentRouter**, responsabile dell'esposizione delle API di backend_G al frontend_G. Questo modulo delega la validazione dei dati in ingresso e uscita agli schemi Pydantic. Questa scelta protegge il core dell'applicazione e lo isola completamente. I metodi principali di questa classe sono:

```

start_analysis(
    request: AssessmentRequestSchema,
    use_case: StartAssessmentUseCase,
) -> AssessmentResponseSchema:

```

Avvia l'esecuzione della pipeline_G ricevendo in input l'IP_G e le informazioni di contesto del dispositivo target. Poiché la produzione del report è un'operazione che richiede diversi minuti per completarsi, la funzione non ritorna il report finale, ma lancia il processo in *background* e ne ritorna l'ID. In questo modo il frontend_G può effettuare un *polling* periodico per monitorare lo stato dell'operazione e mostrare all'utente il suo avanzamento.

```
get_status(
    analysis_id: str,
    use_case: GetAssessmentStatusUseCase
) -> PipelineSchema:
```

Dato l'ID di un'analisi, ne ritorna lo stato sotto forma di **PipelineSchema** che contiene

- stato
- step della pipeline
- messaggio descrittivo
- eventuale errore o *warning*

```
get_analysis_report(
    analysis_id: str,
    use_case: GetAssessmentReportUseCase
) -> VulnerabilityReportSchema:
```

Dato l'ID di un'analisi, ne ritorna, se esistente e pronto, il report finale sotto forma di **VulnerabilityReportSchema**, tra i campi principali contiene una lista di vulnerabilità con le seguenti informazioni:

- CVE_G
- la severità calcolata dallo scanner (nell'MVP Qualys_G)
- CVSS_G
- EPSS_G
- la presenza nel KEV_G
- la severità calcolata tramite il framework di Shimizu e Hashimoto [1] (*Vulnerability Management Chaining*)
- un resoconto generato dall'AI_G

```
export_analysis_report(
    request: ExportRequestSchema,
    analysis_id: str,
    use_case: ExportAssessmentReportUseCase
) -> Response:
```

Dato l'ID di un'analisi e l'estensione richiesta (nell'MVP l'unico formato disponibile è DOCX_G), ne ritorna il file scaricabile.

7.1.3. Servizi Applicativi

7.1.3.1. AssessmentApplicationService

L'**AssessmentApplicationService** è l'orchestratore principale dell'applicazione. Gestisce i tre casi d'uso principali, organizzando tutto il ciclo di vita del software.

```
start_assessment(request: AssessmentRequest) -> AssessmentResponse:
```

Crea l'analisi e chiama in *background* la pipeline_G di esecuzione, in questo modo il backend_G non è bloccato durante l'esecuzione e può gestire altre richieste da parte del frontend_G. Ritorna al frontend_G l'ID dell'analisi creata, affinché questo possa utilizzarlo per chiederne lo stato e recuperare il report.

```
get_status(self, analysis_id: str) -> Pipeline:
```

Utilizza l'ID della pipeline_G per recuperare da una memoria volatile la **StoredAnalysis** contenente l'analisi.

7.1.3.2. PriorityEngine

Il **PriorityEngine** è una classe di supporto all'**AssessmentApplicationService** che incapsula la logica di calcolo della priorità di ThreatLens. Questa classe modella il *decision tree* descritto da Shimizu e Hashimoto [1].

7.1.4. Outbound Adapters

7.1.4.1. Scanner

Per l'MVP è stato codificato un adapter per lo scanner di vulnerabilità Qualys_G, ma il sistema, grazie alla sua architettura, è aperto a nuovi scanner tramite la codifica di adapter dedicati. L'adapter deve implementare il metodo della porta:

```
scan(ip: str) -> ScannerResult
```

Tale metodo riceve l'IP_G del dispositivo target e ritorna l'oggetto di dominio **ScannerResult**, nel quale vengono mappati i risultati provenienti dalle API_G del tenant di Qualys_G.

7.1.4.2. AnalysisStore

L'adapter **InMemoryAnalysisStore** gestisce la persistenza volatile delle **StoredAnalysis**, contenenti i risultati di un'analisi prodotti al termine della pipeline_G. Mette a disposizione metodi di lettura e scrittura dell'oggetto:

```
get_analysis(analysis_id: str) -> StoredAnalysis
```

Ritorna l'oggetto desiderato tramite il suo ID

```
get_all_analysis() -> list
```

Ritorna tutte le analisi salvate nel sistema

```
save_analysis(analysis: StoredAnalysis) -> None:
```

Salva un'analisi nel sistema

7.1.4.3. Data providers

Il recupero dei dati necessari al calcolo della gravità (CVSS_G, EPSS_G, KEV_G), viene gestito da tre classi che si occupano di interrogare API di servizi esterni e normalizzare la loro risposta in oggetti di dominio.

- **NvdCvssAdapter** presenta il metodo `get_cvss_bulk(cve_list: list[str]) -> dict[str, CvssData]` che ritorna dal *database* di NVD_G i valori CVSS_G delle CVE_G che gli sono state fornite.

- `FirstEpssAdapter` espone il metodo `get_epss_bulk(cve_list: list[str]) -> dict[str, EpssData]` che ritorna dal *database* del `FIRSTG` i valori `EPSSG`.
- `CisaKevAdapter` ha il metodo `check_kev_bulk(cve_list: list[str]) -> dict[str, KevData]` che indica per ogni `CVEG` se sia presente nel catalogo del `CISAG`.

7.1.4.4. AiAdapter

Il `GeminiExplanationAdapter` concretizza l'interfaccia definita in `AiExplanationPort`. Il modulo implementa il metodo:

```
generate_explanation_bulk(
    vulnerabilities: list[PrioritizedVulnerability],
) -> ExplanationById
```

Questo metodo riceve in input l'output delle fasi precedenti della *pipeline_G*: una lista di vulnerabilità già prioritzate e arricchite con le relative metriche di contesto. Restituisce una struttura dati indicizzata (`ExplanationById`) che mappa l'identificativo di ciascuna vulnerabilità al resoconto testuale generato dall'Intelligenza Artificiale.

7.2. Principi di Design e Modularità nel backend_G

Durante la progettazione sono state usate una serie di tecniche e design pattern volti a risolvere specifiche sfide implementative. L'applicazione di queste soluzioni ha permesso di strutturare il sistema in modo solido ed efficace, garantendo maggiore modularità e manutenibilità.

7.2.1. Inversione delle dipendenze

Il principio di **inversione delle dipendenze** (*Dependency Inversion Principle*) rappresenta il fondamento dell'architettura esagonale perché rende possibile il disaccoppiamento tra modellazione del dominio e tecnologie esterne. In `ThreatLens` il nucleo applicativo, composto dalle classi di dominio e i servizi applicativi, non importa nessuna libreria esterna o tecnologia, ma fa uso solamente di moduli nativi del linguaggio Python. I servizi applicativi, per ottenere i dati necessari al calcolo, non eseguono direttamente chiamate `APIG`, ma fanno riferimento a interfacce astratte (le *outbound ports*) che espongono dei metodi generici (per esempio `fetch_data()` o `scan()`) la cui implementazione sarà gestita da un modulo esterno al *core*. Questo approccio ha tre principali vantaggi:

- **Alta testabilità:** per testare un servizio applicativo non è necessario istanziare l'infrastruttura reale, ma basterà iniettare un componente fittizio (un *mock_G*).
- **Isolamento degli errori:** la maggior parte delle criticità in un sistema deriva dall'interazione con tecnologie esterne (es. *timeout* di rete, deserializzazione di *payload_G* imprevisti o librerie che possono variare e diventare incompatibili con il nostro sistema). Isolando in un modulo esterno è possibile gestire in modo più efficace e ordinato questi errori, senza inquinare internamente la logica del sistema.
- **Modularità:** se si vuole cambiare tecnologia basta scrivere un altro adapter dedicato, senza dover modificare la logica interna del sistema. Questa flessibilità si è rivelata utile anche in fase di sviluppo, consentendo lo sviluppo del *core* tramite adattatori *dummy* (es. oggetti che ritornano risposte fittizie simulando le `APIG` esterne) per verificare il funzionamento interno del sistema e man mano integrare le tecnologie esterne con moduli reali.

7.2.2. Estensibilità tramite Strategy e Registry

L'architettura è stata ideata per garantire l'estensibilità verso molteplici strumenti di diagnostica (come `QualysG` o `NessusG`), in modo da poter confrontare i risultati provenienti da più scanner. Per risolvere questo problema di design è stato adottato il pattern **Strategy**: ogni adattatore concreto implementa l'interfaccia `ScannerPort`, incapsulando la propria logica di scansione.

La selezione dinamica dello scanner avviene a *run-time*, in base al parametro `ScannerType` fornito nella richiesta dell'utente. Questa responsabilità è dello `ScannerRegistryAdapter` che concretizza l'interfaccia `ScannerRegistryPort` implementando il pattern *Registry*. A differenza dei pattern creazionali come il *Factory*, che gestiscono l'istanziamento di nuovi oggetti, il *Registry* opera esclusivamente come risolutore di dipendenze. Lo `ScannerRegistryAdapter` riceve tramite *Dependency Injection* istanze di adattatori già configurate e create all'avvio dell'applicazione dal *Composition Root* (rappresentato nel sistema dal file `dependencies.py` di FastApiG).

7.2.3. Single Responsibility Principle

Al fine di mantenere un codice ordinato e manutenibile, senza avere funzioni lunghe e complesse che gestiscono diverse parti del sistema, è necessario applicare rigorosamente il *Single Responsibility Principle* (SRP). L'SRP impone un vincolo architetturale più profondo: un modulo, una classe o una funzione deve avere uno e un solo motivo per essere modificato.

- **A livello di classe:** la classe `AssessmentApplicationService` agisce esclusivamente da orchestratore del caso d'uso, ma delega l'effettiva esecuzione di tutte le sotto-operazioni ad altri moduli del sistema. Ad esempio utilizza un adapter esterno per ottenere la lista di `CVEG` che passerà ad altri moduli, oppure non calcola direttamente la priorità operativa ma delega l'operazione al `PriorityEngine`.
- **A livello di funzione:** il metodo `_run_pipeline` gestisce con modularità tutte le fasi della `pipelineG`, delegando l'esecuzione ad altri metodi specializzati nell'operazione (`_scan` o `_calculate_priority`). In questo modo ciascun metodo isola una tipologia di fallimento dell'operazione e un motivo di cambiamento distinto.

Questa rigorosa compartimentazione garantisce che modifiche future o la gestione di errori specifici non inquinino il flusso principale dell'applicazione, favorendo una gestione granulare delle eccezioni (tramite `PipelineFailure` specializzate) e garantendo un'elevata testabilità del codice.

7.3. Modellazione della `pipelineG` di assessment

La `pipelineG` di assessment è il motore del sistema che colleziona dati da una serie di servizi esterni e da questi ne calcola un report finale ordinato. La classe che orchestra queste operazioni è l'`AssessmentApplicationService` che fa partire l'analisi con il metodo `start_assessment`. Il processo della `pipelineG` richiede diversi minuti, dunque è stato gestito attraverso l'utilizzo dei *thread* di Python. Nel thread viene avviato in *background* il metodo `run_pipeline` che consta di cinque fasi:

1. scansione
2. enrichment
3. calcolo della priorità
4. generazione della spiegazione con l'`AIG`
5. creazione del report

Nell'implementazione si è cercato di rispettare il Single Responsibility Principle, infatti come si può notare dalla struttura della `pipelineG`, nelle classi c'è un metodo principale orchestratore che chiama una serie di metodi privati che eseguono una sola operazione logica. Questo rende il codice più leggibile e manutenibile, cercando di atomizzare le operazioni di una funzione, dando più semantica ed evitando funzioni ingestibili con centinaia di righe di codice, favorendo inoltre testabilità e gestione degli errori.

7.3.1. Fase 1: Scansione

Viene gestita dalla funzione `_scan` che utilizza la `ScannerRegistryPort` per recuperare lo scanner selezionato e la `ScannerPort` per ottenere un oggetto `ScannerResult`, contenente la lista di *finding* rilevati.

7.3.2. Fase 2: Enrichment

La logica di questa fase risiede nella funzione `_enrich_with_data` che preleva la lista di `CVEG` dall'oggetto `ScannerResult` creato nella fase precedente. In sequenza viene data questa lista di vulnerabilità ai tre data provider:

- `_cvss_provider`
- `_epss_provider`
- `_kev_provider`

che con i loro metodi `_get_*_bulk` (ogni provider al posto di `*` ha il dato che ricerca) arricchiscono la `CVEG` con i dati per il calcolo. Ne risulta una lista di `EnrichedVulnerability` che contiene degli elementi indicizzati per `CVEG` con i dati che ne indicano la gravità.

7.3.3. Fase 3: Calcolo della priorità

Questa fase è affidata al metodo `_calculate_priority`. La computazione è delegata al componente `PriorityEngine`, che incapsula la logica del *decision tree_G*.

L'elaborazione restituisce una lista di `PrioritizedVulnerability`. Ad ogni vulnerabilità viene assegnata una `OperationalPriority` che ne categorizza la gravità in ordine crescente:

- TRACK
- TRACK*
- ATTEND
- ACT

Questo modulo calcola inoltre con la funzione `_calculate_context_attention` la `ContextAttention` tramite la `_CONTEXT_ATTENTION_MATRIX` che prende in input i tre parametri selezionati all'inizio dell'analisi dall'utente.

7.3.3.1. Fase 4: Generazione della spiegazione con l'`AIG`

Dopo aver ottenuto tutti i dati necessari all'analisi di una `CVEG`, l'`AIG` ha il compito di produrre una spiegazione sintetica in linguaggio naturale, con l'obiettivo di fornire un chiaro contesto della situazione motivando la priorità operativa assegnata dal `PriorityEngine`. La logica di questa fase risiede nel metodo `_generate_ai_explanation` che ha la responsabilità di utilizzare il metodo dell'`AiExplanationPort`, gestendone correttamente la risposta. Per una questione di performance, nell'`MVPG` vengono analizzate solamente le prime cinque vulnerabilità più gravi. Queste vengono passate all'`_ai_explanation_provider` che tramite il metodo `generate_explanation_bulk` genera le spiegazioni necessarie a costruire la lista di `ExplainedVulnerability`.

7.3.3.2. Fase 5: Creazione del report

Come ultima fase vi è la creazione di un report riassuntivo. Il metodo `_generate_report` ha il compito di aggiornare lo stato della pipeline_G a `COMPLETED` e salvare in memoria l'oggetto finale, il `VulnerabilityReport`, con un identificativo grazie al quale sarà possibile recuperarlo e mostrarlo all'utente.

7.3.4. Architettura di gestione degli errori

Al fine di tradurre in modo granulare eccezioni provenienti dall'esterno, facendole fluire nel modo corretto fino all'interfaccia utente, il sistema implementa un'architettura di gestione degli errori strutturata e centralizzata. Questo non solo permette di averne una gestione ordinata, ma anche di isolare nei punti corretti la logica di gestione delle eccezioni, in modo da non inquinare il dominio con la logica riguardante le tecnologie esterne. La logica di strutturazione è stata individuare le possibili classi di errore del sistema e costruirci una gerarchia di errori che potesse incapsulare anche i diversi tipi di errore provenienti da servizi esterni.

In principio l'architettura suddivide le eccezioni nelle seguenti categorie principali:

- **Eccezioni Applicative (`ApplicationError`)**

- Fallimenti di pipeline_G (**PipelineError**)
- Guasti Infrastrutturali (**InfrastructureError**)

7.3.4.1. Eccezioni Applicative

Rappresentano condizioni previste da casi d'uso del sistema. Ognuna è associata a un codice di errore standardizzato (**ErrorCode**). Le eccezioni applicative del sistema implementano la generica **ApplicationError** (che a sua volta implementa la classe **Exception**) e sono:

- **AnalysisNotFoundError**: segnala che l'analisi ricercata non è stata trovata
- **ReportNotReadyError**: segnala che il report non è ancora stato generato
- **UnsupportedScannerError**: segnala che la richiesta di scanner non è supportata dal sistema
- **UnsupportedExportedExtensionError**: segnala che l'estensione richiesta non è supportata dal sistema
- **ExportError**: segnala un errore durante l'esportazione

7.3.4.2. Fallimenti di pipeline_G

Rappresentano errori bloccanti che si verificano durante il flusso sequenziale della pipeline_G. Queste eccezioni hanno un codice di errore che registra lo specifico step di esecuzione in cui il sistema si è arrestato, permettendo di identificare con precisione il punto di rottura. I fallimenti della pipeline_G implementano la generica **PipelineFailure** (che a sua volta implementa la classe **Exception**) e sono:

- **ScanFailure**: segnala un errore durante la fase di scansione.
- **PriorityCalculationFailure**: segnala un errore durante la fase di calcolo interno della priorità.
- **ReportBuildFailure**: segnala un errore durante la fase di costruzione del report.
- **UnexpectedPipelineFailure**: errore generico per errori non contemplati nel corso della pipeline_G.

7.3.4.3. Guasti infrastrutturali

Modellano i fallimenti derivati dagli outbound adapters. Queste classi hanno il compito di tradurre le eccezioni sollevate dalle librerie di terze parti o dalle API_G esterne in errori gestibili dal sistema. Solitamente API e librerie esterne possono ritornare una vasta gamma di errori differenti, per questo sono stati gestiti i casi di errore principali o che sono stati ritenuti rilevanti avendone fatta esperienza in fase di sviluppo. Questi errori vengono incapsulati nelle classi di errori di sistema più ampie con annessa una descrizione esplicativa. L'impatto applicativo di questi errori è delegato ai livelli superiori del sistema. I guasti infrastrutturali implementano il generico **InfrastructureError** (che a sua volta implementa la classe **Exception**) e sono:

- **ProviderUnavailableError**: il provider non è raggiungibile o non risponde.
- **ProviderTimeoutError**: il provider non ha risposto entro il timeout.
- **ProviderAuthenticationError**: il provider ha rifiutato le credenziali.
- **ProviderRateLimitError**: il provider ha applicato un limite alle richieste.
- **ProviderResponseError**: la risposta ricevuta non rispetta il formato atteso.
- **PersistenceError**: errore durante lettura o scrittura della persistenza.
- **FileGenerationError**: errore tecnico durante la generazione di un file.

7.3.4.4. Traduzione verso l'interfaccia

La responsabilità di tradurre le eccezioni interne in risposte HTTP_G è delegata ad appositi *exception handlers*. Questo approccio garantisce:

- **Standardizzazione del contratto API_G**: ogni errore esposto all'utente segue un **ErrorResponseSchema** che include:
 - un codice identificativo
 - un messaggio descrittivo
 - dettagli tecnici opzionali

- **Mappatura semantica dei codici:** gli errori applicativi vengono tradotti nei corretti codici HTTP_G, come **404 Not Found** per una risorsa inesistente o **409 Conflict** per conflitti di stato applicativo.
- **Gestione della validazione:** gli errori generati da input non conformi vengono restituiti con codice **422 Unprocessable Content**.
- **Tracciamento e sicurezza:** eccezioni inattese non vengono espone in chiaro all'utente, ma il sistema restituisce un errore interno generico (**500 Internal Server Error**) e registra la traccia dell'eccezione (*stack trace*) tramite i log. Questo facilita le operazioni di debug senza compromettere la sicurezza del sistema.

7.4. Architettura di frontend_G

Il frontend_G è stato progettato adottando un'architettura a livelli (*Layered Architecture*) al fine di favorire una rigorosa separazione delle responsabilità. Sebbene nella pratica comune di sviluppo dell'ecosistema Angular si faccia talvolta riferimento al pattern *Model-View-ViewModel* (MVVM), a livello architetturale la struttura implementata si fonda sull'integrazione del pattern *Presentation Model* con un livello di astrazione basato su *Facade*. Questa struttura è stata adottata dopo un'attenta analisi delle moderne *best practice* consolidate all'interno della *community* di sviluppatori Angular. Tali pattern, ampiamente discussi e validati nei canali specializzati di settore, rappresentano oggi uno standard emergente per la scalabilità e manutenibilità di applicazioni web reattive.

Questa scomposizione garantisce che l'interfaccia utente sia completamente disaccoppiata dalle complessità di rete, dalla logica di dominio e dall'orchestrazione dei flussi asincroni. Il sistema è pertanto strutturato in tre macro-livelli:

- **Presentation Layer:** Organizzato secondo il pattern *Smart e Dumb components*, in cui i componenti presentazionali (View) gestiscono esclusivamente il rendering, mentre i componenti contenitore (Smart) adattano lo stato alle esigenze della vista.
- **Abstraction Layer:** Mediato da un'implementazione reattiva del pattern *Facade*, che non si limita a fornire un'interfaccia unificata, ma orchestra i flussi asincroni e funge da singola fonte di verità per lo stato della UI.
- **Core Layer:** Composto da servizi API rigorosamente *stateless* che operano come *Gateway* verso il backend, rispettando il principio di singola responsabilità.

7.4.1. Service Model

Questi moduli hanno la responsabilità di comunicare con le API_G di backend_G, tramite chiamate HTTP_G. Secondo i principi della programmazione reattiva, i metodi di queste classi non ritornano un oggetto statico, ma un canale dinamico dal quale è possibile «osservare» i dati richiesti.

7.4.1.1. AnalysisApi

Si occupa della gestione di un'analisi di Vulnerability Assessment_G, presenta i metodi:

```
startAnalysis(AssessmentRequest): Observable<AssessmentResponse>
```

Metodo che lancia la creazione dell'analisi e riceve un *Observable* di tipo **AssessmentResponse** contenente l'identificativo del processo lanciato.

```
getAnalysesSummary(): Observable<AnalysesSummary>
```

Metodo che ritorna l'ID e stato di tutte le analisi salvate nella memoria del sistema, serve a visualizzare nella home la lista di analisi create.

7.4.1.2. CapabilitiesApi

Si occupa di recuperare le funzionalità di cui il sistema dispone. Serve a recuperare le opzioni selezionabili nel modulo di configurazione dell'analisi (come scanner disponibili e opzioni di contesto dell'asset). Questa classe è particolarmente utile perché rende il frontend_G intelligente e dipendente dalle funzionalità codificate nel backend_G: nel caso si aggiunga un nuovo scanner non sarà necessario modificare il frontend_G, essendo lui stesso a rilevare un nuovo scanner e mostrandone automaticamente l'opzione disponibile. Questo modulo è un buon esempio di come nel sistema siano le tecnologie esterne a dipendere da logica e configurazioni interne. Il metodo di questa classe è:

```
getCapabilities(): Observable<SystemCapabilities>
```

Ritorna all'utente le funzionalità di cui il sistema dispone.

7.4.1.3. PipelineApi

Servizio che si occupa di richiedere lo stato della pipeline_G di un'analisi, con il metodo:

```
getStatus(analysisId): Observable<Pipeline>
```

Ritorna un *Observable* di tipo **Pipeline** (un oggetto di frontend_G) contenente le informazioni della pipeline_G che verranno mostrate all'utente.

7.4.1.4. ReportApi

Ha la responsabilità di gestire operazioni riguardanti il report: richiederlo per mostrarlo all'utente e richiederne l'esportazione.

```
getReport(analysisId): Observable<VulnerabilityReport>
```

Ritorna un *Observable* di tipo **VulnerabilityReport**, contenente tutte le informazioni da mostrare all'utente.

7.4.2. Facade

Questo layer ha la responsabilità di sollevare lo *Smart Component* dalla complessa gestione dei flussi di dati reattivi basata sugli *Observable*. Questi canali di comunicazione asincrona richiedono un'orchestrazione attenta e centralizzata. Mischiare tale logica con la gestione dei *Dumb Component* avrebbe generato una classe difficilmente manutenibile. Delegando la gestione degli *Observable* alla *Facade*, si rispetta il *Single Responsibility Principle*, mantenendo il livello di presentazione pulito e focalizzato sulle logiche dell'interfaccia.

7.4.2.1. HomeFacade

Questo modulo si occupa di gestire lo stato della *Home*, la pagina principale di ThreatLens. Presenta un singolo metodo:

```
loadAnalyses(): void
```

Utilizza **analysisApi** per caricare le analisi nella home, gestendone lo stato ed eventuali errori.

7.4.2.2. NewAnalysisFacade

Questa classe ha il compito di gestire lo stato della pagina di analisi, inclusa la pipeline_G visiva, presenta due metodi:

```
loadCapabilities(): void
```

Utilizza **capabilitiesApi** per caricare le funzionalità del sistema, gestendone stato ed eventuali errori.

```
startAnalysis(AssessmentRequest): void
```

Utilizza **analysisApi** per lanciare l'analisi; resta in ascolto sul canale di risposta attendendo l'identificativo del processo generato. Si occupa anche di far partire il metodo privato **startPollingStatus(analysisId)** che chiede periodicamente lo stato del processo, in modo da monitorare l'andamento della pipeline_G di backend_G, informando l'utente del suo stato e ridirezionando l'interfaccia alla pagina di report una volta terminata l'esecuzione.

7.4.2.3. ReportFacade

Il **ReportFacade** orchestra lo stato della pagina di report, con i metodi:

```
loadReport(analysisId): void
```

Utilizza **reportApi** per recuperare il report e gestire il flusso di dati ed eventuali errori.

```
exportReport(analysisId, format): void
```

Utilizza **reportApi** per generare e recuperare il file esportabile nel formato selezionato dall'utente.

7.4.3. Smart Component (ViewModel)

Questo layer ha il compito di gestire la logica di interfaccia e passare i dati già elaborati ai componenti figli. Deve inoltre restare in ascolto degli eventi emessi dall'utente, per invocare correttamente i metodi della facade lasciandole la responsabilità di orchestrare la chiamata di rete. Solitamente in Angular lo Smart Component rappresenta una pagina web contenente vari pezzi di interfaccia con cui l'utente può interagire (i Dumb Component). Dunque, gli Smart Component di ThreatLens rappresentano le pagine della piattaforma:

- **HomePage**
- **NewAnalysisPage**
- **ReportPage**

7.4.4. Dumb Component (View)

Rappresentano il livello di presentazione puro. La loro unica responsabilità è presentare gli elementi dell'interfaccia utente e delegare l'interazione dell'utente «verso l'alto», notificando lo Smart Component tramite l'emissione di eventi. Essendo completamente privi di logica applicativa e ignari dei servizi API_G o della Facade, questi componenti lavorano esclusivamente sui dati ricevuti in ingresso, risultando pertanto altamente riutilizzabili.

7.4.5. Flussi Asincroni e Gestione dello Stato Reattivo

Nell'ambito dello sviluppo di interfacce web moderne, la gestione degli eventi asincroni e dei flussi di dati continui rappresenta una sfida architeturale di primaria importanza. In ThreatLens è stato deciso di adottare un paradigma ibrido, in modo da realizzare un sistema moderno, senza aumentare troppo la complessità. Le operazioni di rete prolungate vengono orchestrate tramite la libreria *RxJS*, mentre la propagazione dello stato all'interfaccia utente è demandata al moderno costruito dei *Signal*.

Per ottenere le informazioni sullo stato della pipeline_G non è stata implementata una comunicazione bidirezionale persistente con il backend, per via della sua complessità. Per implementare

una feature visiva come l'andamento della `pipelineG` è stato ritenuto sufficiente delegare al client la responsabilità di verificare attivamente gli aggiornamenti nel server di `backendG`. Questa problematica è stata risolta implementando il pattern del *Polling Consumer*:

- **Il ruolo dell'Orchestratore (Facade):** questo modulo incapsula la logica di *polling*, interrogando attivamente e periodicamente il backend. Ogni volta che riceve un aggiornamento (ad esempio l'esito della scansione o l'avanzamento della `pipelineG`), non emette eventi tramite i classici *Subject* tipici del pattern Observer canonico, ma aggiorna direttamente il proprio stato reattivo sincrono tramite i *Signal*. In questo modo la Facade agisce come fonte di verità per dati in sola lettura verso l'interfaccia.
- **Il ruolo del Consumatore (Smart Component):** il *ViewModel* agisce da osservatore, senza conoscere la logica di rete. Non gestisce la chiamata HTTP iterativa, ma si limita a reagire passivamente alla lettura dei *Signal* esposti dalla Facade. Non appena quest'ultima aggiorna il proprio stato, il componente viene notificato automaticamente e propaga i dati aggiornati ai *Dumb Component* per mostrarli all'utente.

Questa rigorosa separazione garantisce che il ciclo di vita della richiesta rimanga confinato nel livello della Facade, offrendo al livello di presentazione un'interfaccia dichiarativa e reattiva, costantemente sincronizzata con il server.

Capitolo 8.

Implementazione e Scelte Tecnologiche

8.1. Modellazione e Validazione dei Dati

Avendo scelto un'architettura di tipo esagonale e FastApi_G come framework per il backend_G, è stato necessario scegliere in quale punto del sistema utilizzare Pydantic_G (libreria per la validazione dati spesso usata, poiché integrata nativamente, con FastApi_G). Consultando diversi forum di *developers*, è emerso che inserire Pydantic_G in un'architettura esagonale è un problema concreto che anima diverse discussioni. Le due scelte plausibili sono:

1. creare nel dominio delle classi Pydantic_G
 2. mantenere Pydantic_G all'esterno del dominio, costruendolo con le *dataclasses* di Python_G
- Gli argomenti a sostegno della prima tesi sono una minore quantità di codice *boilerplate*, evitando duplicazioni inutili e mappature di dati che aggiungono logica non necessaria al funzionamento del programma, allungando il lavoro del programmatore. Tuttavia, la seconda tesi è maggiormente supportata da gran parte della letteratura architeturale. In primo luogo, introdurre una libreria esterna come Pydantic_G nel dominio violerebbe il principio dell'architettura esagonale o della *clean architecture*, come riportato dalla letteratura [8]. In secondo luogo, la validazione dei dati deve avvenire ai confini del sistema: nel nostro caso validiamo i dati in entrata e in uscita nel layer dell'Inbound Adapter. Dunque, la soluzione è stata scrivere delle funzioni di mappatura a livello dell'Inbound Adapter, in modo che i dati vengano validati e serializzati in un oggetto Pydantic_G che possa rispettare le richieste del contratto dell'API_G. In questo modo, oltre che validare, è possibile mantenere la logica di dominio stabile e lasciare eventuali modifiche al livello di mappatura dell'oggetto, adattandolo alle esigenze delle API_G.

8.2. Persistenza dei dati nell'MVP_G

Nell'MVP (*Minimum Viable Product*) è stato deciso di mantenere una persistenza dati volatile, senza implementare un rigoroso *database*. L'adattatore esterno **InMemoryAnalysisStore** gestisce autonomamente la persistenza e recupero degli oggetti tramite semplici metodi **get** e **save** che simulano un *database* salvandoli in un dizionario. Tuttavia, in questo modo gli oggetti alla distruzione dell'istanza della classe (ad esempio quando il container dell'app viene ricostruito), vengono dimenticati e non è più possibile recuperarli. Questa scelta è stata fatta per semplificare e velocizzare lo sviluppo, considerando che i requisiti dell'attuale prototipo non rendono strettamente necessaria l'integrazione di un sistema completo per la gestione di basi di dati. Ad ogni modo, grazie alla modularità dell'architettura esagonale, una futura transizione verso una soluzione di persistenza stabile risulterebbe un'operazione semplice. Per integrare un *database* sarà sufficiente sviluppare un nuovo *Outbound Adapter* che concretizzi i metodi già definiti dalle interfacce delle porte dedicate. Questo approccio assicura che il nuovo innesto non alteri altre parti della piattaforma, come la logica interna di backend_G o la parte di interfaccia di frontend_G.

8.3. Sicurezza e comunicazione HTTP

L'architettura del sistema prevede una rigorosa segregazione tra il livello di presentazione (Angular, servito sulla porta **4200**) e il livello applicativo (FastAPI, esposto sulla porta **8000**). Anche se questa separazione garantisce un'elevata modularità e previene l'esposizione di informazioni

sensibili lato client, introduce un vincolo nella comunicazione diretta a causa della *Same-Origin Policy* (SOP).

La SOP è una regola di sicurezza implementata dai browser che impedisce agli script eseguiti in una specifica «origine» (formata dalla combinazione di protocollo, dominio e porta) di leggere dati provenienti da un'origine differente. Nel nostro caso le origini di frontend e backend differiscono nella porta, per questo si attiva tale regola di sicurezza, bloccando di default le richieste dirette dal frontend_G al backend_G.

Per comunicare al sistema che questa origine differente risulta sicura, occorre implementare il protocollo *Cross-Origin Resource Sharing* (CORS). In FastAPI_G la gestione del CORS viene condotta dal *Middleware*, una copertura esterna attraverso la quale passa ogni richiesta al server.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:4200"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

In questo modo, il server è configurato per consentire l'origine differente del frontend (**http://localhost:4200**), consentendo qualsiasi metodo e qualsiasi header. Questo funziona perché quando Angular_G con *HttpClient* effettua una richiesta verso FastAPI_G, il browser esegue prima una *Preflight Request* di questo tipo:

```
OPTIONS /api/test-connezzione HTTP/1.1
Host: 127.0.0.1:8000
Origin: http://localhost:4200
Access-Control-Request-Method: GET
```

chiedendo al server se autorizza questa chiamata. Grazie al *Middleware* il server è configurato per accettarla, dunque non riceve un messaggio di errore e viene effettuata la richiesta, rispondendo al *Preflight* con una risposta tipo:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:4200
Access-Control-Allow-Methods: GET, POST, OPTIONS
```

8.4. Richieste API_G Batch

Una sfida implementativa è stata doversi scontrare con i limiti imposti dalle API_G di tecnologie esterne, in particolare i data providers e il modello gratuito di Gemini. Infatti, non è possibile, per limiti infrastrutturali di queste tecnologie, richiedere con una sola chiamata API_G dati o informazioni per centinaia di CVE_G senza incorrere in un *rate limit*_G. Per risolvere questa problematica è

stato necessario dividere il payload totale di CVE_G in batch la cui dimensione non raggiungesse il *rate limit_G* imposto dalla specifica API_G . In certi casi, tra una chiamata di rete e l'altra è anche stato necessario introdurre un ritardo programmato per distanziarle temporaneamente, sempre per rispettare i limiti imposti dalle tecnologie esterne. Questa logica è stata confinata al livello degli *Outbound Adapters*, rivelando ancora una volta i vantaggi di modularità dell'architettura esagonale e di isolamento del dominio rispetto alle tecnologie esterne.

Capitolo 9.

Integrazione dell'Intelligenza Artificiale

L'obiettivo dell'integrazione di un Modello Linguistico all'interno di ThreatLens non è delegare un calcolo del rischio classificando la CVE_G , ma rielaborare tutti i dati ricavati dai vari provider in una sintesi in linguaggio naturale fornendo un chiaro contesto all'analista di sicurezza. All'intelligenza artificiale è stata data la direttiva di agire come un analista di *cybersecurity* senior.

Per ottenere risultati affidabili, coerenti e privi di allucinazioni, è stata strutturata in input una direttiva che utilizza tecniche di *Prompt Engineering* con un'infrastruttura di validazione dell'output rigorosa.

9.1. Strutturazione del Contesto e prompt_G Engineering

Il prompt_G inviato all' LLM_G non è una semplice stringa testuale, ma un payload_G JSON strutturato che incapsula in modo ordinato tutte le metriche elaborate dalla pipeline_G . A seguito di attente ricerche è emerso che il formato più leggibile da un LLM_G è il JSON, perché consente di strutturare il prompt_G in chiavi-valori che aiutano il modello nella comprensione della direttiva e aumentano le performance di risposta in output. Il prompt_G risiede in un file dedicato che è composto dalle seguenti sezioni:

- **Detection Evidence:** Contiene le evidenze puramente tecniche rilevate dallo scanner (es. Qualys_G) sullo specifico asset. Il modello è istruito a distinguere rigorosamente tra ciò che è stato effettivamente osservato (host, porte, stato della rilevazione) e la documentazione generale della vulnerabilità fornita dal vendor.
- **CVE Evidence:** Include le metriche oggettive associate alla specifica CVE_G (punteggio CVSS_G , stima EPSS_G e presenza nel catalogo KEV_G). Il prompt_G fornisce direttive esplicite su come interpretare questi segnali: ad esempio, l'assenza della vulnerabilità dal catalogo del KEV_G non indica sistematicamente che questa non sia mai stata sfruttata.
- **ThreatLens Assessment:** Rappresenta il risultato calcolato dal **PriorityEngine**. Al modello viene vietato di ricalcolare, correggere o alterare la priorità calcolata dal sistema, limitandone il ruolo alla sola spiegazione delle motivazioni che hanno portato a tale risultato.

9.2. Analisi dell'Allineamento

Una delle criticità principali è stata la frequente discordanza tra la gravità assegnata dallo scanner enterprise di terze parti e la priorità calcolata dal motore interno. Per gestire questa problematica, il sistema calcola a priori una metrica definita *Alignment Observation*, la quale confronta quantitativamente le due valutazioni incasellandole in categorie predefinite (es. valutazioni allineate verso l'alto, oppure rischio dello scanner elevato ma priorità calcolata inferiore). Questa classificazione viene passata al modello AI con regole rigorose: in caso di divergenza, l'IA deve spiegare in linguaggio naturale la differenza osservabile chiarendo che i due sistemi valutano metriche differenti, senza mai presentare tale discordanza come un errore tecnico. In questo modo verrà posta l'attenzione dell'analista sui motivi di disallineamento delle due metriche fornendo, tramite questo confronto, ulteriore contesto all'analisi.

9.3. Vincoli di Output e Prevenzione delle allucinazioni_G

Affinché la risposta sia utilizzabile dal frontend_G, il sistema impone che l'output finale sia privo di formattazioni esterne (come il Markdown).

La struttura dell'output è rigidamente tipizzata e richiede la generazione di campi specifici per ogni vulnerabilità analizzata:

- **Summary:** Una sintesi interpretativa.
- **Detection Context & Priority Rationale:** La contestualizzazione tecnica e la spiegazione della priorità.
- **Alignment Explanation:** La spiegazione dell'eventuale disallineamento tra lo scanner e ThreatLens.
- **Documented Action:** Le azioni di mitigazione concrete, estraendo solo dati reali senza inventare comandi o procedure non documentate.
- **Uncertainty:** L'esplicitazione formale del limite informativo più rilevante per il caso in esame.

Per limitare ulteriormente il fenomeno delle allucinazioni_G, il prompt_G include una serie di limiti comportamentali. Al modello sono stati imposti dei divieti come affermare che un asset sia compromesso in assenza di prove o introdurre informazioni esterne non presenti nel payload_G di partenza.

Capitolo 10.

Conclusioni

Durante il periodo di tirocinio è stato possibile acquisire e consolidare diverse competenze, che spaziano dalla classificazione delle vulnerabilità alla progettazione e allo sviluppo di soluzioni software full-stack. La complessità del dominio ha richiesto uno studio approfondito delle metriche di valutazione delle minacce, rendendo necessaria la consultazione di autorevoli *paper* scientifici al riguardo. Nella fase di sviluppo è stato inoltre possibile interfacciarmi con strumenti enterprise come Qualys_G, permettendo di comprendere le dinamiche di analisi e gestione delle vulnerabilità.

Dal punto di vista implementativo, la realizzazione del prodotto ha comportato l'adozione di tecnologie moderne e architetture scalabili. Per sviluppare il frontend_G è stato necessario studiare il framework di Angular_G, in questo modo ho avuto l'occasione di padroneggiare il funzionamento di un moderno strumento per la creazione di interfacce web reattive. Il backend_G è stato strutturato secondo i paradigmi dell'architettura esagonale e sviluppato impiegando FastApi_G, una delle tecnologie di riferimento per la realizzazione di API performanti.

L'interazione con sfide architetture e operative concrete, come la configurazione delle policy CORS, la gestione di rate limit_G e l'ottimizzazione dei processi in presenza di risorse hardware e software limitate, ha conferito al lavoro un forte pragmatismo, riducendo la distanza tra la teoria accademica e la pratica industriale.

Per concludere, l'inserimento in un contesto lavorativo dinamico e strutturato ha rappresentato un grande valore aggiunto, poiché il costante confronto con professionisti competenti ha arricchito significativamente questo percorso di crescita tecnica e personale.

Capitolo 11.

Glossario

- AI** *Artificial Intelligence* - sistema automatizzato, dotato di un certo grado di autonomia, progettato per operare in vista di obiettivi espliciti o impliciti, capace di generare, a partire dai dati in input, risultati sotto forma di previsioni, contenuti, raccomandazioni o decisioni, influenzando ambienti reali o virtuali. Nel presente testo, il termine è usato in senso estensivo per indicare l'impiego di modelli LLM come Gemini Flash nella generazione di contenuti testuali a partire da dati in input.
- API** *Application Programming Interface* - insieme di regole e protocolli che consente a sistemi software distinti di comunicare tra loro, astruendo e nascondendo i dettagli implementativi tramite un contratto formale.
- Angular** Framework open-source multiplatforma sviluppato e mantenuto da Google (insieme a una community globale) per costruire applicazioni web dinamiche, scalabili e manutenibili, basate principalmente su TypeScript.
- CISA** La *Cybersecurity and Infrastructure Security Agency* (CISA) è un'agenzia federale statunitense parte del Dipartimento della Sicurezza Interna responsabile per la sicurezza informatica e delle infrastrutture su suolo statunitense.
- CPE** Le Common Platform Enumeration sono stringhe standardizzate utilizzate per identificare in modo univoco hardware, software e sistemi operativi all'interno di un'infrastruttura.
- CTEM** Il Continuous Threat Exposure Management è un framework di cybersecurity secondo il quale non basta solo scoprire e ordinare le vulnerabilità, ma bisogna continuamente identificare, prioritizzare e validare in base al rischio reale e al contesto di business. Questo è un processo continuo di scoping, discovery, prioritization, validation e mobilization.
- CVE** Le Common Vulnerabilities and Exposures sono un dizionario pubblico di vulnerabilità e falle di sicurezza note.
- CVSS** *Common Vulnerability Scoring System* - standard sviluppato da FIRST per valutare la gravità delle vulnerabilità di sicurezza, assegnando un punteggio da 0,0 a 10,0.
- Claude** Avanzato modello linguistico (LLM) sviluppato da Anthropic.
- DOCX** formato di file predefinito per Microsoft Word, utilizzato per archiviare documenti di elaborazione testi contenenti testo formattato, immagini, tabelle e altri elementi multimediali.
- EPSS** *Exploit Prediction Scoring System*, gestito da FIRST, stima la probabilità che una CVE venga sfruttata nei successivi 30 giorni.
- FIRST** *Forum of Incident Response and Security Teams*: organizzazione globale leader nella risposta agli incidenti di sicurezza, che riunisce oltre 800 team di esperti per la cooperazione internazionale.
- FastApi** Moderno framework web per Python utilizzato per costruire API ad alte prestazioni.
- HTTP** *HyperText Transfer Protocol* - protocollo applicativo usato come principale sistema per la trasmissione d'informazioni su internet.
- IP** *Internet Protocol* - identificativo di un dispositivo all'interno di una rete.
- KEV** Il CISA KEV è un catalogo di vulnerabilità note già sfruttate attivamente, indicando se l'exploit scoperto è già stato sfruttato.
- LLM** *Large Language Model* - sistema avanzato di intelligenza artificiale basato sul deep learning e su architetture di reti neurali. Questi modelli vengono addestrati su enormi quantità di dati testuali per comprendere, interpretare e generare linguaggio naturale in modo coerente e contestualmente appropriato.
- MVP** *Minimum Viable Product* - versione più semplice e basilare di un prodotto o servizio, dotata solo delle funzionalità essenziali per essere utilizzata dai primi utenti.

NVD Il *National Vulnerability Database* è un archivio gestito dal governo degli Stati Uniti e in particolare dal *National Institute of Standards and Technology (NIST)* che raccoglie informazioni sulle vulnerabilità di sicurezza note.

Nessus Software proprietario di tipo client-server sviluppato da Tenable Network Security, uno degli scanner di vulnerabilità più usati in ambito industriale.

Notion Applicazione web di produttività e gestione degli appunti, sviluppata da Notion Labs Inc. Lanciata nel 2016, Notion offre funzionalità per la creazione e l'organizzazione di note, documenti, database, bacheche Kanban e molto altro.

Prompt Engineering Processo strategico di progettazione, perfezionamento e ottimizzazione degli input (chiamati prompt) forniti ai modelli di intelligenza artificiale, per guidarne la produzione di output accurati, pertinenti e di alta qualità.

Pydantic Libreria Python open source per la validazione, la serializzazione e la trasformazione dei dati, basata sull'uso delle annotazioni di tipo (type hints) native del linguaggio.

Python Linguaggio di programmazione ad alto livello, interpretato e open source.

Qualys Piattaforma enterprise utilizzata per il Vulnerability Management, permette alle aziende di identificare, classificare e monitorare le falle di sicurezza all'interno di prodotti IT.

SDK *Software Development Kit* - indica genericamente un insieme di strumenti per lo sviluppo.

SSVC *Stakeholder-Specific Vulnerability Categorization* - Metodologia ad alberi decisionali per prioritizzare le vulnerabilità, adattando le azioni di risposta allo specifico contesto operativo degli stakeholder.

Teams Piattaforma Microsoft di comunicazione e collaborazione unificata che combina chat di lavoro persistente, teleconferenza e condivisione di contenuti.

TypeScript Linguaggio di programmazione con tipizzazione statica opzionale, spesso utilizzato nella realizzazione di applicazioni web.

UML Standard per la modellazione visiva di sistemi software, usato nel progetto per creare i diagrammi delle classi.

Vulnerability Assessment Processo sistematico di identificazione, quantificazione e classificazione delle vulnerabilità in un sistema.

Vulnerability Fatigue / Alert Fatigue Stato di esaurimento mentale e disimpegno psicologico che colpisce gli operatori della sicurezza informatica e gli sviluppatori, derivante dall'essere sommersi da un volume eccessivo di vulnerabilità e alert. Questo fenomeno porta a una desensibilizzazione verso i rischi reali, poiché la difficoltà di gestire tutte le minacce individuate porta a ignorare o rimandare indefinitamente la risoluzione delle falle, aumentando la superficie di attacco aziendale.

allucinazioni Nel campo dell'intelligenza artificiale è il fenomeno in cui il modello genera un output con argomentazioni false o inventate, presentandole come verità oggettive.

asset context Insieme di parametri (quali ambiente, esposizione e criticità) che definiscono il profilo di rischio di un asset informatico. Consente di contestualizzare le vulnerabilità rilevate valutandone il potenziale impatto operativo reale.

backend Parte di un'applicazione o di un sito web che gestisce la logica di business, il database e altre operazioni invisibili all'utente finale.

cloud Rete di server accessibili tramite internet, utilizzati per eseguire applicazioni e archiviare dati senza l'uso di macchine fisiche locali.

codebase Il termine codebase, o code base, è usato nello sviluppo del software per indicare l'intera collezione di codice sorgente usata per costruire una particolare applicazione o un particolare componente.

core Parte interna dell'architettura esagonale, include dominio, service e porte di inbound ed outbound per interfacciarsi con le tecnologie esterne.

debugging Indica l'attività che consiste nell'individuazione e correzione da parte del programmatore di uno o più errori (bug) rilevati nel software.

decision tree Un *decision tree* (albero decisionale) è un modello logico composto da **nodi decisionali** (che valutano una condizione sui dati in ingresso), **rami** (che instradano il flusso in base all'esito) e **nodi foglia** (che definiscono il verdetto finale).

exploit Codice o procedura che sfrutta attivamente una vulnerabilità di un sistema informatico per comprometterne la sicurezza o il funzionamento.

frontend Componente visiva e interattiva di un'applicazione software con cui l'utente si interfaccia direttamente. Gestisce la presentazione dei dati e l'interfaccia grafica, operando tipicamente lato client e separando la visualizzazione dalla logica di backend.

mock Oggetti fittizi che imitano il comportamento di oggetti reali in modo controllato. Vengono utilizzati principalmente negli unit test per isolare il codice testato, simulando dipendenze complesse, non deterministiche o non ancora implementate (come database o API esterne).

patch Piccolo pezzo di codice o un file eseguibile progettato per aggiornare, correggere o migliorare un programma, un sistema operativo o il firmware.

patching Processo di risoluzione di un problema in modo reattivo.

payload Porzione di un messaggio, pacchetto o trasmissione che contiene i dati effettivi destinati all'utente finale o all'applicazione.

pentesting Pratica di sicurezza informatica che consiste nell'esecuzione di attacchi simulati e autorizzati contro sistemi, reti o applicazioni per identificare e sfruttare le vulnerabilità che un cybercriminale potrebbe utilizzare.

pipeline Catena di trasformazioni automatizzate che gestisce il ciclo di vita del software o un flusso dei dati.

prompt Nel campo dell'intelligenza artificiale è il messaggio in input (direttiva) che viene fornito al modello linguistico.

rate limit Tecnica di controllo che limita il numero di richieste che un utente, client o indirizzo IP può inviare a un server o a un'API entro un intervallo di tempo specifico.

refactoring Il refactoring è una tecnica di ingegneria del software che consiste nella ristrutturazione della struttura interna del codice sorgente con l'obiettivo principale di migliorare la leggibilità, la manutenibilità e l'efficienza del codice, rendendolo più pulito e facile da comprendere per gli sviluppatori.

remediation Fase operativa e strutturata dedicata alla correzione definitiva delle vulnerabilità identificate durante la scansione.

scanner termine che indica scanner di vulnerabilità (come Qualys) che consentono di rilevare le CVE associate alle CPE di un dispositivo.

stakeholder Persona, gruppo o organizzazione direttamente o indirettamente coinvolto nel progetto e in grado di influenzarne o subirne le conseguenze sull'esito finale.

tenant Cliente o organizzazione che dispone di uno spazio privato e isolato all'interno di un sistema software condiviso.

triage Nel campo del vulnerability assessment indica il processo di classificazione di vulnerabilità in classi di emergenza crescenti in base alla loro gravità.

Bibliografia

- [1] N. Shimizu e M. Hashimoto, «Vulnerability Management Chaining: An Integrated Framework for Efficient Cybersecurity Risk Prioritization», *IEEE Access*, 2026.
- [2] Angular Team, «Angular Overview». Consultato: 10 settembre 2026. [Online]. Disponibile su: <https://angular.dev/overview>
- [3] Qualys, Inc., «Qualys API (VM and PA) User Guide». settembre 2026. [Online]. Disponibile su: <https://cdn2.qualys.com/docs/qualys-api-vm-pa-user-guide.pdf>
- [4] S. Rajamani, «CVSS + EPSS Combined Vulnerability Prioritisation», *Journal of Cybersecurity*, 2025.
- [5] C. Fruhwirth e T. Mannisto, «Improving CVSS-based vulnerability prioritization and response with context information», in *2009 3rd International Symposium on Empirical Software Engineering and Measurement*, 2009.
- [6] Cybersecurity and Infrastructure Security Agency (CISA), «Stakeholder-Specific Vulnerability Categorization (SSVC)». Consultato: 1 settembre 2026. [Online]. Disponibile su: <https://www.cisa.gov/resources-tools/resources/stakeholder-specific-vulnerability-categorization-ssvc>
- [7] K. G. Agyei *et al.*, «Explainable Risk-Based Vulnerability Prioritization in Hybrid Cloud: Integrating CVSS, EPSS, and CISA KEV with Asset Criticality Signals», *World Journal of Advanced Research and Reviews*, 2026.
- [8] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.